Optimizing Key Recovery in Impossible Cryptanalysis and Its Automated Tool

Haoyang Wang and Jianing Zhang

School of Computer Science, Shanghai Jiao Tong University, Shanghai, China.

*Corresponding author(s). E-mail(s): haoyang.wang@sjtu.edu.cn;
Contributing authors: zhangjn@sjtu.edu.cn;

Abstract

Key recovery for impossible differential (ID) cryptanalysis has seen few systematic advancements in nearly a decade, while the recent resurgence of impossible boomerang (IB) attacks has produced several powerful but disparate strategies. This, combined with the high complexity of existing tools for finding IB distinguishers, highlights the need for a unified optimization framework. In this paper, we address these gaps by introducing the Impossible Upper and Lower Boomerang Connectivity Tables (`iUBCT`/`iLBCT`), new lightweight tools for finding two-round contradictions. We then modernize key recovery by providing the first formal analysis of probabilistic extensions and by adapting the generic key guessing framework from rectangle cryptanalysis, which enables a systematic search for attacks with optimal complexities, particularly for memory.

These techniques are integrated into a new Mixed-Integer Linear Programming (MILP)-based tool, leading to a suite of improved attacks on SKINNY, SKINNYee, Midori, and Deoxys-BC. Our results include the first 30-round attack on SKINNYee, extending the state of the art by one round. For other targets, our framework yields attacks with substantial memory and time reductions. In particular, we reduce the complexity of the best previous ID attack on SKINNY-128-384 by factors of up to $\mathbf{2^{131}}$ (memory) and $\mathbf{2^{9}}$ (time), and the best previous IB attack on Deoxys-BC-384 by factors of up to $\mathbf{2^{56}}$ (memory) and $\mathbf{2^{25}}$ (time). Many of these results provide new time-memory-data trade-offs, often achieving complexity reductions at the cost of moderately increased data requirements.

Keywords: Impossible differential cryptanalysis, impossible boomerang cryptanalysis, SKINNY, SKINNYee, Midori, Deoxys-BC

1 Introduction

Impossible differential (ID) [1, 2] and impossible boomerang (IB) [3] cryptanalysis are powerful techniques for evaluating the security of block ciphers. As variants of differential cryptanalysis [4], they leverage zero-probability events to discard incorrect key candidates, often leading to some of the best-known attacks against modern cryptographic primitives. Research in this field has historically progressed along two complementary lines: the construction of potent distinguishers and the development of efficient key recovery attacks.

For ID cryptanalysis, distinguishers are typically found using the *miss-in-the-middle* technique [5], a process that has been significantly enhanced by the development of automated search tools. These have evolved from early Mixed-Integer Linear Programming (MILP)-based models [6, 7] to Constraint Programming (CP)-based models capable of handling related-key scenarios [8].

Progress in the key recovery phase, however, has been notably slower. Foundational techniques like the early-abort method [9] and the formal complexity framework by Boura et al. [10, 11] established the state of the art nearly a decade ago. While recent automated tools can now model the entire attack, including key recovery [12–14], the underlying key recovery techniques have seem fewer systematic advances. A recent work by Song et al. [15] incorporated the meet-in-the-middle technique into ID key recovery, though its utility is typically limited to ciphers where the key size is at least twice the block size. Despite these developments, two powerful approaches that have been highly successful in related-field cryptanalysis remain underexplored in the context of impossible attacks. First, while *probabilistic extensions* have been used sporadically in early ID attacks [9, 16–18], they have never been formally analyzed. Boura et al. [10] briefly mentioned the potential of applying probabilistic extensions to AES-like ciphers, but did not provide any further analysis. The recent systematic application of this technique in rectangle attacks by Song et al. [19] has shown its potential, but its impact on the key-elimination paradigm of impossible cryptanalysis remains an open question. Second, the *arbitrary key guessing strategies* that have become standard for optimizing key recovery in differential and rectangle attacks [20–24] have not been adapted to the ID framework, leaving attack complexities potentially far from optimal.

The IB attack, first introduced by Lu [3, 25], saw a significant resurgence of interest recently [26–29], leading to new distinguisher construction methods and automated search tools. Despite this renewed attention, finding powerful, multi-round IB distinguishers remains a formidable challenge. The standard method of using the Boomerang Connectivity Table (BCT) is effective for contradictions over a single round [26, 27], but extending this is nontrivial. Initial attempts to use the Double Boomerang Connectivity Table (DBCT) for two-round contradictions were shown to be flawed [26, 30], while corrected versions like the DBCT* are often too computationally expensive to be practical for use in automated searches [30]. Furthermore, key recovery for IB attacks is still in its infancy, with only a few specific, non-generic methods proposed so far [26, 27].

Our contributions. To address the above limitations and advance the state of the art, this paper introduces a multi-faceted approach to optimize impossible cryptanalysis. Our main contributions are fourfold:

- We introduce two new tools, the *Impossible Upper Boomerang Connectivity Table* (`iUBCT`) and *Impossible Lower Boomerang Connectivity Table* (`iLBCT`) , which efficiently detect the majority of two-round contradictions for IB distinguishers while overcoming the computational and modeling bottlenecks of existing methods.
- We provide the first formal analysis of probabilistic extensions in impossible cryptanalysis, demonstrating that it introduces a trade-off that can reduce time and memory complexities at the cost of increased data requirements.
- We adapt and formalize a generic key recovery framework that supports arbitrary key guessing strategies, enabling a systematic search for attacks with substantially lower memory complexity.
- We integrate these advances into a MILP-based tool for the automated search of full ID and IB attacks.

To demonstrate the power of our methods, we applied them to several widely-analyzed ciphers, yielding a series of new and improved attacks on SKINNY, SKINNYee, Midori, and Deoxys-BC. Our results present enhanced attacks and new trade-offs between complexities. Notably, we obtain a new 19-round IB distinguisher for SKINNY-128-384, extending the previous state of the art by one round. We present the first 30-round related-tweak IB attack on SKINNYee, breaking one more round than the previous best attack. For many existing attacks, we achieve significant reductions in memory and time complexities, sometimes at the cost of a moderate increase in data complexity. A summary of our main results and a comparison with previous work is presented in Table 1.

Outline. The remainder of this paper is organized as follows. Section 2 provides background on impossible cryptanalysis. Section 3 introduces our new techniques for identifying IB distinguishers. In Section 4, we formally analyze probabilistic extensions. Section 5 details our generic key recovery framework. Our automated tool is described in Section 6, with applications presented in Section 7. We conclude in Section 8.

2 Preliminaries

2.1 Notations

Impossible cryptanalysis, including ID and IB attacks, exploits differential/boomerang distinguishers with a probability of 0 to eliminate incorrect key candidates. An attack is constructed by first identifying an impossible distinguisher, denoted as E_{D}, which spans several internal rounds of a cipher. This distinguisher establishes that a specific input difference Δ_{I} can never propagate to a specific output difference Δ_{O}.

As illustrated in Figure 1, a key recovery attack is mounted by extending this distinguisher over additional rounds: a backward extension E_{B} and a forward extension E_{F}. The attacker collects data (pairs for ID attacks, quartets for IB attacks) that conform to the expected plaintext difference space Δ_{B} and ciphertext difference space

Table 1: Overview of some cryptanalytic results. STK=single-tweakey. SK=single-key. RT=related-tweak. RTK=related-tweakey. Int.=integral. MITM=Meet-in-the-Middle. Boom.=Boomerang. Rect=Rectangle. $^{(*)}$ = other types of attacks achieving the best results. $^{(\dagger)}$ = complexity corrected by us.

Target	#R	Data	Time	Memory	Attack	Setting	Ref.
SKINNY-64-64	19	$2^{61.47}$	$2^{63.03}$	2^{56}	ID	RTK	[31]
	19	$2^{61.47}$	$\mathbf{2^{62.76}}$	$\mathbf{2^{52}}$	ID	RTK	L.2
SKINNY-128-128	19	$2^{122.47}$	$2^{124.60}$	2^{112}	ID	RTK	[31]
	19	$2^{122.47}$	$\mathbf{2^{124.43}}$	$\mathbf{2^{104}}$	ID	RTK	L.2
SKINNY-64-128	19	$2^{60.86}$	$2^{110.34}$	2^{104}	ID	STK	[13]
	19	$2^{65.05}$	$\mathbf{2^{104.90}}$	$\mathbf{2^{68.05}}$	ID	STK	L.3
	22	2^{58}	2^{106}	2^{104}	Int.$^{(*)}$	SK	[14]
SKINNY-128-256	19	$2^{117.86}$	$2^{219.23}$	2^{208}	ID	STK	[13]
	19	$2^{126.05}$	$\mathbf{2^{209.45}}$	$\mathbf{2^{133.05}}$	ID	STK	L.3
	22	2^{114}	2^{213}	2^{208}	Int.$^{(*)}$	SK	[14]
SKINNY-64-192	21	$2^{62.43}$	$2^{174.42}$	2^{168}	ID	STK	[13]
	21	$2^{64.99}$	$\mathbf{2^{169.38}}$	$\mathbf{2^{103.99}}$	ID	STK	L.4
	26	2^{62}	2^{166}	2^{164}	Int.$^{(*)}$	SK	[14]
SKINNY-128-384	21	$2^{122.89}$	$2^{347.35}$	2^{336}	ID	STK	[13]
	21	$2^{125.99}$	$\mathbf{2^{338.65}}$	$\mathbf{2^{204.99}}$	ID	STK	L.4
	26	2^{122}	2^{331}	2^{328}	Int.$^{(*)}$	SK	[14]
Midori64	11	2^{60}	$2^{116.59}$	2^{92}	ID	SK	[32]
	11	$2^{61.33}$	$\mathbf{2^{99.94}}$	$\mathbf{2^{56.33}}$	ID	SK	M.2
	12	$2^{55.5}$	$2^{125.5}$	2^{109}	MITM$^{(*)}$	SK	[33]
SKINNYee	29	$2^{67.2}$	$2^{119.2}$	2^{48}	IB	RT	[26]
	30	$2^{67.03}$	$2^{123.61}$	$\mathbf{2^{44}}$	IB	RT	Section 7
Deoxys-BC-256	10	$2^{132.8}$	$2^{186.66}$	$2^{104(\dagger)}$	IB	RTK	[27]
	10	$2^{132.9}$	$\mathbf{2^{177.42}}$	$\mathbf{2^{56}}$	IB	RTK	N.2
	11	$2^{122.4}$	$2^{218.65}$	2^{128}	Boom.$^{(*)}$	RTK	[22]
Deoxys-BC-384	14	$2^{130.9}$	2^{368}	$2^{128(\dagger)}$	IB	RTK	[27]
	14	$2^{132.41}$	$\mathbf{2^{343.05}}$	$\mathbf{2^{72}}$	IB	RTK	N.3
	15	$2^{115.7}$	$2^{371.7}$	2^{128}	Rect.$^{(*)}$	RTK	[19]

Δ_F. For each collected structure, the subkey bits involved in the extensions, k_B and k_F, are guessed. Any key guess that causes the data to align with the impossible (Δ_I, Δ_O) pair is proven incorrect and is consequently discarded. The key parameters used to describe this framework and analyze its complexity are formally defined in Table 2.

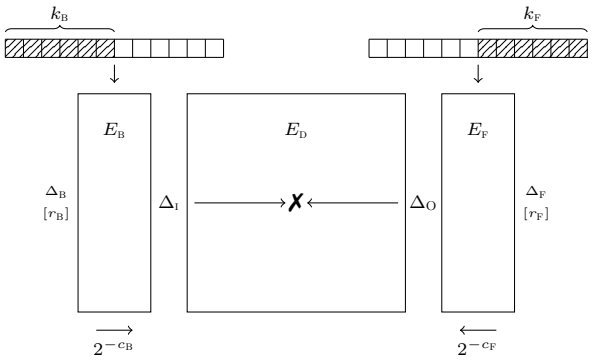


Fig. 1: Outline of impossible cryptanalysis.

Table 2: Notations in impossible cryptanalysis

E_D	impossible distinguisher
Δ_I (resp. Δ_O)	input difference (resp. output difference) of the distinguisher E_D
E_B (resp. E_F)	backward extension from Δ_I (resp. forward extension from Δ_O)
Δ_B (resp. Δ_F)	difference propagated from Δ_I through E_B^{-1} (resp. from Δ_O through E_F)
r_B (resp. r_F)	the dimension of vector space Δ_B (resp. Δ_F)
c_B	the number of bit-conditions that should be verified from Δ_B to Δ_I
c_F	the number of bit-conditions that should be verified from Δ_F to Δ_O
k_B (resp. k_F)	the subkey bits involved in E_B (resp. E_F)

2.2 Impossible Differential Cryptanalysis

Impossible differential cryptanalysis, independently proposed by Knudsen [1] and Biham [2], is a powerful technique that uses a differential with a probability of zero to eliminate incorrect key candidates. The key recovery attack is generally divided into the following phases, with complexities formalized as in [10, 11, 13].

Pairs Generation. The first step is to collect a sufficient number of pairs that could potentially satisfy the input and output differences of the extended distinguisher. The probability that a trial key is kept in the candidate keys is $P = (1 - 2^{-(c_B+c_F)})^N$ for N pairs. To recover g key bits (i.e. $P = 2^{-g}$), the number of pairs is

$$N \approx 2^{c_B+c_F+\mathrm{LG}(g)}$$

where $\mathrm{LG}(g) = \log_2(g) - 0.53$. The data complexity D required to generate these N pairs depends on the structure of the plaintext and ciphertext differences (r_B and r_F), and is given by:

$$D = \max\left\{ \min_{r_X \in \{r_B, r_F\}} \left\{ \sqrt{2^{c_B+c_F+n+1+\mathrm{LG}(g)-r_X}} \right\}, 2^{c_B+c_F+n+1+\mathrm{LG}(g)-r_B-r_F} \right\}$$

Guess-and-Filter. For each of the N pairs, the involved subkey bits $k_{\mathrm{B}} \cup k_{\mathrm{F}}$ are guessed to check if the pair satisfies the impossible distinguisher. Any key guess that results in a valid path would be discarded. The time complexity of this phase can be bounded by

$$T_1 + T_2 = N + 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|} \frac{N}{2^{c_{\mathrm{B}}+c_{\mathrm{F}}}} = 2^{c_{\mathrm{B}}+c_{\mathrm{F}}+\mathrm{LG}(g)} + 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|+\mathrm{LG}(g)} \tag{1}$$

Note that T_2 is a theoretical lower bound, the exact value should be evaluated for each attack using the early-abort technique [34].

Exhaustive Search. After the filtering phase has successfully suggested g bits of the key, the remaining $k - g$ key bits must be found via exhaustive search. The time complexity for this is:

$$T_3 = 2^{k-g}$$

The attack requires storing either the N candidate pairs or the discarded key candidates. The memory complexity is therefore:

$$M = \min\{2^{c_{\mathrm{B}}+c_{\mathrm{F}}+\mathrm{LG}(g)}, 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|}\} \tag{2}$$

This unified framework provides the basis for the classical ID attack, which we enhance in Section 5 with our generic key guessing strategy.

2.3 Impossible Boomerang Cryptanalysis

The impossible boomerang cryptanalysis, first introduced by Lu in [3, 25], combines the concepts of boomerang and impossible differential attacks. It relies on a boomerang distinguisher with a probability of zero, which is formally defined as follows.

Definition 1. Suppose $E : \{0,1\}^n \times \{0,1\}^k \to \{0,1\}^n$ is a block cipher and $K \in \{0,1\}^k$ is a key for E. If there exists a quartet of n-bit blocks $(\alpha, \alpha', \delta, \delta')$ satisfying

$$\forall X \in \mathbb{F}_2^n,\ \Pr[E_K^{-1}(E_K(X) \oplus \delta) \oplus E_K^{-1}(E_K(X \oplus \alpha) \oplus \delta') = \alpha'] = 0,$$

then the combination of $(\alpha, \alpha', \delta, \delta')$ is called an impossible boomerang distinguisher for E, written $(\alpha, \alpha') \nrightarrow (\delta, \delta')$.

As illustrated in Figure 2, the original construction of an IB distinguisher follows a miss-in-the-middle approach [25, 28]. It requires two differentials with probability 1 over the upper sub-cipher E_0 ($\alpha_0 \to \alpha_1$ and $\alpha_0' \to \alpha_1'$) and two over the lower sub-cipher E_1^{-1} ($\beta_2 \to \beta_1$ and $\beta_2' \to \beta_1'$). The contradiction is established if $\alpha_1 \oplus \alpha_1' \oplus \beta_1 \oplus \beta_1' \neq 0$ for all possible propagations. A common approach in recent works [26, 27] is to consider one differential $\alpha_0 \to \alpha_1$ (resp. $\beta_3 \to \beta_2$) with probability 1 for E_0 (resp. E_1), and then identify a contradiction within an intermediate sub-cipher E_m. The simplest case involves a single-round E_m, where the impossibility arises from a zero-probability boomerang switch. The impossibility can be identified by zero entries of the `BCT`.

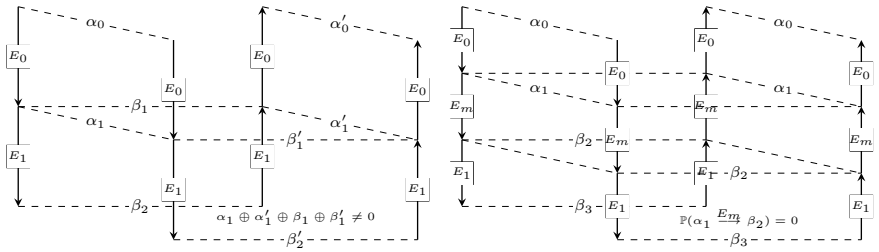


Fig. 2: Constructions of impossible boomerang distinguishers: using four different differentials [25, 28] (left) and a specific case using two different differentials [26, 27] (right).

Definition 2 (BCT [35])**.** Let S be a bijective function over $\mathbb{F}_2^n$, and $\alpha, \beta \in \mathbb{F}_2^n$. The BCT of S is a two-dimensional table defined as:

$$\texttt{BCT}(\alpha, \beta) = \#\{x \in \mathbb{F}_2^n | S^{-1}(S(x) \oplus \beta) \oplus S^{-1}(S(x \oplus \alpha) \oplus \beta) = \alpha\}.$$

For contradictions spanning multiple rounds, early works [26, 27, 30] considered the DBCT [36, 37]. However, this approach was proven to be invalid in most cases. As pointed out in [26, 30], the original DBCT formulation (see Definition 3) implicitly assumes that the intermediate differences for the upper and lower trails are identical (i.e., $\alpha_1 = \alpha'_1$ and $\beta_1 = \beta'_1$), which inevitably excludes valid propagation paths where these differences are unequal. Therefore, it is suggested to use the DBCT variants: the DBCT* [38] or GDBCT [27], both cover all the possibilities of the intermediate differences. These tables are built upon the Upper and Lower BCTs (UBCT/LBCT), which are formally defined in Appendix A.

Definition 3 (DBCT [36, 37])**.** Let S be a bijective function over $\mathbb{F}_2^n$. The DBCT of S is a two-dimensional table defined as

$$\texttt{DBCT}(\alpha, \beta) = \sum_{\alpha_1, \beta_1} \texttt{UBCT}(\alpha, \alpha_1, \beta_1) \cdot \texttt{LBCT}(\alpha_1, \beta_1, \beta).$$

Definition 4 (DBCT* [38])**.** Let S be a bijective function over $\mathbb{F}_2^n$. The DBCT* of S is a two-dimensional table defined as

$$\begin{aligned}\texttt{DBCT}^*(\alpha, \beta) = \sum_{\alpha_1, \alpha'_1, \beta_1, \beta'_1} &\left(\# \left\{ x \in \mathbb{F}_2^n \;\middle|\; \begin{array}{l} S(x) \oplus S(x \oplus \alpha) = \alpha_1, \\ S(x) \oplus \beta_1 \oplus S(x \oplus \alpha) \oplus \beta'_1 = \alpha'_1, \\ S^{-1}(S(x) \oplus \beta_1) \oplus S^{-1}(S(x \oplus \alpha) \oplus \beta'_1) = \alpha \end{array} \right\} \right) \\ \times &\left(\# \left\{ x \in \mathbb{F}_2^n \;\middle|\; \begin{array}{l} x \oplus S^{-1}(S(x) \oplus \beta) = \beta_1, \\ S(x \oplus \alpha_1) \oplus S(x \oplus \alpha_1 \oplus \beta'_1) = \beta, \\ S^{-1}(S(x) \oplus \beta) \oplus S^{-1}(S(x \oplus \alpha_1) \oplus \beta) = \alpha'_1 \end{array} \right\} \right).\end{aligned}$$

Key Recovery. Similar to an ID attack, the key recovery for an IB attack involves extending the distinguisher and filtering key candidates, but it operates on quartets of

data instead of pairs. Two distinct methods for IB key recovery were recently proposed in [26, 27]. A detailed overview of these methods is provided in Appendix D.

3 The Impossible Upper/Lower Boomerang Connectivity Table

Accurately identifying multi-round contradictions is essential for building powerful IB distinguishers. While the DBCT* can precisely analyze a two-round boomerang switch, it suffers from two major limitations. First, the computational complexity is often prohibitive. The theoretical complexity for computing the DBCT* for an n-bit S-box is $\mathcal{O}(2^{5n})$ [26, 38]. In practice, this means computing the table for a single 8-bit S-box can take hours and require huge RAM. Second, encoding the complex constraints of the DBCT* into automated search models is complicated, which significantly reduces search efficiency.

To overcome these hurdles, this section introduces two new and efficient cryptanalytic tools: the iUBCT and iLBCT. These tables are specifically designed to detect the majority of two-round contradictions while being significantly faster to compute and far easier to integrate into automated search frameworks.

3.1 The Impossible Upper/Lower Boomerang Connectivity Table and Its Properties

We first recall the definition of the Generalized Boomerang Connectivity Table (GBCT) [39], which is an extension of the BCT.

Definition 5 (GBCT [39])**.** Let S be a bijective function over $\mathbb{F}_2^n$. The GBCT of S is a four-dimensional table defined as

$$\texttt{GBCT}(\alpha, \alpha'; \beta, \beta') = \#\{x \in \mathbb{F}_2^n | S^{-1}(S(x) \oplus \beta) \oplus S^{-1}(S(x \oplus \alpha) \oplus \beta') = \alpha'\}.$$

As discussed previously, IB distinguishers only concern the zero entries in the DBCT*, the tables iUBCT and iLBCT in Definition 6 can help us capture the subset containing zero entries. Figure 3 shows the basic idea of the two tables.

Definition 6 (iUBCT, iLBCT)**.** Let S be a bijective function over $\mathbb{F}_2^n$. The iUBCT and iLBCT of S are defined as:

$$\texttt{iUBCT}(\alpha, \beta) = \# \left\{ (\alpha_1, \alpha_1') \in (\mathbb{F}_2^n)^2 \;\middle|\; \begin{array}{l} \texttt{DDT}(\alpha, \alpha_1) > 0 \\ \texttt{DDT}(\alpha, \alpha_1') > 0 \\ \texttt{GBCT}(\alpha_1, \alpha_1'; \beta, \beta) > 0 \end{array} \right\},$$

$$\texttt{iLBCT}(\alpha, \beta) = \# \left\{ (\beta_1, \beta_1') \in (\mathbb{F}_2^n)^2 \;\middle|\; \begin{array}{l} \texttt{DDT}(\beta_1, \beta) > 0 \\ \texttt{DDT}(\beta_1', \beta) > 0 \\ \texttt{GBCT}(\alpha, \alpha; \beta_1, \beta_1') > 0 \end{array} \right\}.$$

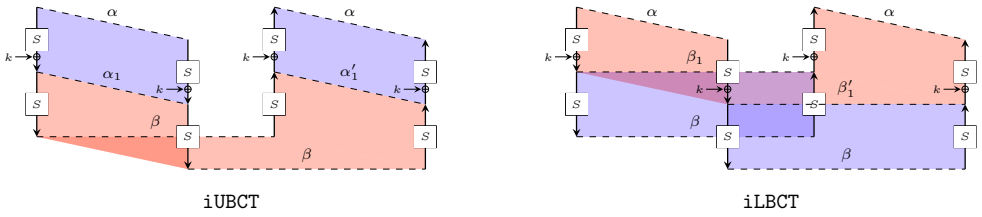


Fig. 3: Framework of iUBCT (left) and iLBCT (right). The blue color describes difference propagation with the effect of the DDT. The red color describes propagation with the effect of the GBCT.

The iUBCT and iLBCT for the SKINNY-64 S-box is shown in Appendix B. Since we only concern zero entries in the iUBCT and the iLBCT, the complexity for identifying the zero entries for an n-bit S-box is $\mathcal{O}(2^{3n})$, which is significantly lower than $\mathcal{O}(2^{5n})$ for the DBCT*. The fast identification algorithm is deferred to Appendix G.

Analogous to the extension of DBCT to 3BCT in [37], we can similarly define i3UBCT/i3MBCT/i3LBCT in Appendix H. Based on these newly proposed cryptanalytic tables and also the BCT, we can enforce

$$\texttt{BCT} = 0/\texttt{iUBCT} = 0 \vee \texttt{iLBCT} = 0/\texttt{i3UBCT} = 0 \vee \texttt{i3MBCT} = 0 \vee \texttt{i3LBCT} = 0$$

to construct contradictions for consecutive 1/2/3 rounds in IB distinguishers.

Below, we present several properties on the iUBCT and iLBCT that directly follow from the definition. A detailed proof is given in Appendix C.

Property 1. For any α, β, we have $\texttt{iUBCT}(0,\beta) = 1$, $\texttt{iLBCT}(\alpha,0) = 1$, $\texttt{iUBCT}(\alpha,0) > 0$ and $\texttt{iLBCT}(0,\beta) > 0$.

Property 2. For a bijective function S over $\mathbb{F}_2^n$ and its inverse S^{-1}, the value of $\texttt{iUBCT}_S(\alpha,\beta)$ is equal to the value of $\texttt{iLBCT}_{S^{-1}}(\beta,\alpha)$. Similarly, the value of $\texttt{iLBCT}_S(\alpha,\beta)$ is equal to the value of $\texttt{iUBCT}_{S^{-1}}(\beta,\alpha)$.

The following property shows that the zero entries of the iUBCT and iLBCT implies zero entries of the DBCT*.

Lemma 1. *For any nonzero α and β, if either* $\texttt{iUBCT}(\alpha,\beta) = 0$ *or* $\texttt{iLBCT}(\alpha,\beta) = 0$*, or both entries are 0, then* $\texttt{DBCT}^*(\alpha,\beta) = 0$.

Proof. Assume we have $\texttt{iUBCT}(\alpha,\beta) = 0$. If $\texttt{DBCT}^*(\alpha,\beta) > 0$, it means there exists at least a pair of values satisfy the boomerang transition of the two rounds. Thus, we can obtain the corresponding differences α_1 and α_1' for this pair, which contradicts the assumption. The proof is analogous for iLBCT. □

However, the converse is not necessarily true: $\texttt{DBCT}^*(\alpha,\beta) = 0$ does not imply that neither $\texttt{iUBCT}(\alpha,\beta) = 0$ or $\texttt{iLBCT}(\alpha,\beta) = 0$, as explained in the following.

Coverage and Limitations of the iUBCT/iLBCT. Our proposed tables, iUBCT and iLBCT, are specifically designed to detect impossibility that is intrinsic to either the

upper or lower propagation trail of the two-round boomerang switch. A result of $\texttt{iUBCT}(\alpha, \beta) = 0$ proves an impossibility due to a failure to find any pair of intermediate differences (α_1, α_1') in the upper trail, while $\texttt{iLBCT}(\alpha, \beta) = 0$ proves one for (β_1, β_1') in the lower trail. However, our method ignores the case that valid propagation paths exist for both the upper and lower trails when considered independently, but for every valid upper trail path, no compatible lower trail path exists. Since $\texttt{DBCT}^*$ considers $(\alpha_1, \alpha_1', \beta_1, \beta_1')$ at the same time, it can cover all the cases. Figure 4 illustrates these scenarios conceptually.

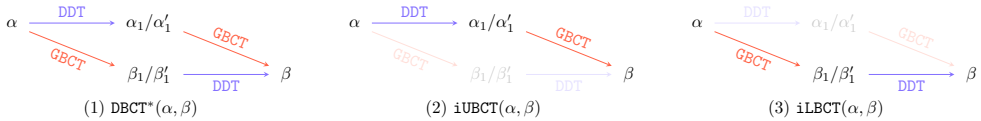


Fig. 4: Comparison between $\texttt{DBCT}^*$ and $\texttt{iUBCT}/\texttt{iLBCT}$

To quantify the proportion of impossible transitions captured by our tables, we analyzed the SKINNY-128 S-box (see Table 3). Our tables identify 84+231 = 315 out of 566 total impossible transitions, corresponding to approximately 55.7%. Furthermore, the iUBCT captures the unique zero entry of $\texttt{DBCT}^*$ for ASCON.

Table 3: Number of zero entries for the $\texttt{DBCT}^*/\texttt{iUBCT}/\texttt{iLBCT}$ for the S-boxes in SKINNY, ASCON and AES.

S-box		$\texttt{DBCT}^*$	iUBCT	iLBCT	iUBCT ∪ iLBCT
4-bit	SKINNY-64	0	0	0	0
5-bit	ASCON	1	1	0	1
8-bit	AES	0	0	0	0
	SKINNY-128	566	84	231	315

While our approach is not exhaustive, it is highly effective in capturing the majority of impossible boomerang behaviors. First, the computational complexity is reduced from $\mathcal{O}(2^{5n})$ to $\mathcal{O}(2^{3n})$. Second, automated models employing the iUBCT and iLBCT are more efficient than those based on $\texttt{DBCT}^*$. In particular, modeling with the iUBCT and iLBCT requires fewer parameters than with the $\texttt{DBCT}^*$, thereby reducing the number of difference states that need be constrained in the automated model and resulting in a smaller model size.

3.2 Application to SKINNY-128-384

To demonstrate the effectiveness of our proposed tools, we present a 19-round related-tweakey IB distinguisher for SKINNY-128-384 [40], derived using our automated tool based on the iLBCT in Section 6. This distinguisher exceeds one more round than the previous one based on the BCT [27].

Here, we only explain the two-round contradiction identified by the iLBCT, as shown in Figure 5. The full distinguisher is provided in Appendix I, and the specification of SKINNY is referred to Appendix L.1.

We apply the iLBCT to Figure 5 with input difference $\alpha = \Delta X_9[15]$ and output difference $\beta = \nabla Y_{10}[2]$, where α and β are determined by the tweakey differences. For the lower part, we derive $\nabla Y_9[15] = \nabla X_{10}[2] \oplus \nabla X_{10}[14] = \nabla X_{10}[2]$, by inverting the MixColumns and ShiftRows operations. We denote by β_1 and β_1' the values of $\nabla Y_9[15]$ for the two lower trails, respectively. The contradiction is then identified by the failure to find any pair of (β_1, β_1') in the lower trail:

$$\texttt{iLBCT}(\alpha, \beta) = \#\{(\beta_1, \beta_1') \mid \texttt{GBCT}(\alpha, \alpha; \beta_1, \beta_1') > 0,\ \texttt{DDT}(\beta_1, \beta) > 0,\ \texttt{DDT}(\beta_1', \beta) > 0\} = 0.$$

The set of admissible values of (α, β) is provided in Table 8.

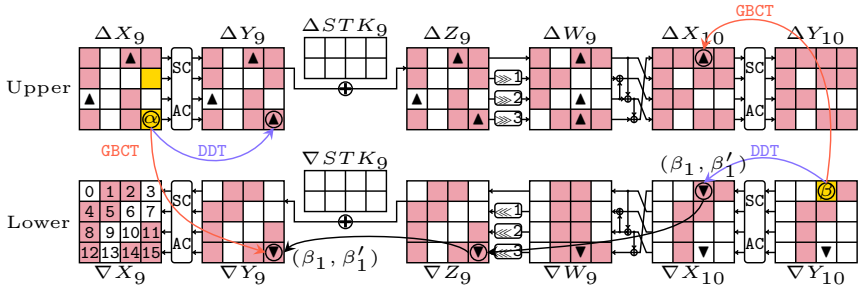


Fig. 5: The two-round contradiction of SKINNY-128-384 based on the iLBCT. The color ▭ denotes an unknown difference and ▭ denotes a known difference. ▼ denotes the bytes required for computing $\texttt{iLBCT}(\alpha, \beta)$, ▲ and ▼ denote the ones for $\texttt{DBCT}^*(\alpha, \beta)$. The cell order is defined in ∇X_9.

It would be computationally difficult to obtain the result in Figure 5 using an automated tool based on the $\texttt{DBCT}^*$. By definition, the $\texttt{DBCT}^*$ considers the propagation of the GBCT and DDT across both S-box layers. Consequently, the number of bytes to be considered increases significantly due to the intermediate linear layer. These extra bytes, denoted by ▲, impose many additional constraints on the automated model throughout the search.

4 Probabilistic Extensions in Impossible Cryptanalysis

Key recovery extensions in classical differential-type cryptanalysis are typically deterministic, propagating differences with probability 1. Recently, Song et al. [19] investigated the use of *probabilistic extensions* in rectangle and differential attacks, where the extension occurs with a probability less than one. Earlier, Boura et al. [10] briefly mentioned this idea for ID attacks, suggesting that it could provide greater

flexibility in selecting trade-off parameters. However, this technique has not been systematically analyzed in the context of impossible cryptanalysis.

While the core idea is analogous, its application to impossible differential-type cryptanalysis is nontrivial due to a fundamental difference in the key recovery paradigm: impossible cryptanalysis relies on the elimination of incorrect keys, whereas standard differential attacks rely on counting right pairs to identify the correct key. This conceptual divergence, together with differing complexity frameworks, necessitates a dedicated and formal investigation of the impact of probabilistic extensions.

This section provides the first such analysis for ID attacks. We will formally derive the changes to the attack complexities and compare them to the classical deterministic approach. The analysis for IB attacks is analogous and therefore omitted for brevity.

4.1 Analysis of Data Complexity

In ID attacks, the data complexity is intrinsically determined by the parameters of the key recovery extension, expressed as $D = 2^{c_{\mathrm{B}}+c_{\mathrm{F}}-r_{\mathrm{B}}-r_{\mathrm{F}}+\mathrm{LG}(g)+n+1}$. Assuming a fixed backward extension E_{B}, the change in data complexity is determined by the term $c_{\mathrm{F}} - r_{\mathrm{F}}$ for the forward extension E_{F}. The following analysis relies on the well-known fundamental relationship for a one-round truncated differential $\Delta X_i \xrightarrow{\overrightarrow{p}_i} \Delta X_{i+1}$:

$$\overrightarrow{p}_i \cdot |\Delta X_i| = \overleftarrow{p}_i \cdot |\Delta X_{i+1}|, \tag{3}$$

where X_i (resp. X_{i+1}) denotes the first (resp. the last) internal state in round i, while $\overrightarrow{p}_i$ and $\overleftarrow{p}_i$ denote the forward and backward probabilities, respectively.

Let the t-round forward extension E_{F} starts from round r. The total number of bit-conditions c_{F} is the sum of the per-round conditions $c_{\mathrm{F}} = \sum_{i=r}^{r+t-1} c_i$. The number of bit-conditions c_i for round i quantifies the filtering power, which is equivalent to the negative logarithm of the backward probability for that round: $c_i = -\log_2(\overleftarrow{p}_i)$. For example, in the deterministic extension of Figure 6, we have $c_r = 16$, $c_{r+1} = 96$, and $c_{r+2} = 0$, yielding $c_{\mathrm{F}} = 112$.

According to Equation (3), we can express c_i using the forward probability:

$$c_i = -\log_2\left(\frac{\overrightarrow{p}_i \cdot |\Delta X_i|}{|\Delta X_{i+1}|}\right) = -\log_2(\overrightarrow{p}_i) + \log_2|\Delta X_{i+1}| - \log_2|\Delta X_i|.$$

Summing c_i over all t rounds of the extension yields c_{F}:

$$\begin{aligned} c_{\mathrm{F}} = \sum_{i=r}^{r+t-1} c_i &= -\sum_{i=r}^{r+t-1} \log_2(\overrightarrow{p_i}) + \sum_{i=r}^{r+t-1} (\log_2|\Delta X_{i+1}| - \log_2|\Delta X_i|) \\ &= -\log_2(\prod_i \overrightarrow{p}_i) + \log_2|\Delta X_{r+t}| - \log_2|\Delta X_r| \\ &= -\log_2(\overrightarrow{p}) + r_{\mathrm{F}} - \log_2|\Delta X_r|, \end{aligned}$$

where $\overrightarrow{p}$ is the total probability of the forward extension and $r_{\mathrm{F}} = \log_2 |\Delta X_{r+t}|$ by definition. Thus, we have

$$c_{\mathrm{F}} - r_{\mathrm{F}} = -\log_2(\overrightarrow{p}) - \log_2 |\Delta X_r|. \tag{4}$$

Since ΔX_r is the output difference of the impossible distinguisher, meaning a constant factor for the attack, the data complexity D is inversely proportional to the probability $\overrightarrow{p}$ of the extension:

$$D = \frac{1}{\overrightarrow{p}} \cdot 2^{-\log_2 |\Delta X_r| + c_{\mathrm{B}} - r_{\mathrm{B}} + n + 1 + \mathrm{LG}(g)} = \frac{1}{\overrightarrow{p}} \cdot D',$$

where D' is the data complexity for deterministic extension in E_{F}. The analysis for the backward extension E_{B} is analogous.

4.2 Analysis of Time and Memory Complexities

The time complexity of the guess-and-filter phase and the memory complexity are primarily determined by the size of the involved subkeys, $|k_{\mathrm{B}} \cup k_{\mathrm{F}}|$, and the total number of bit conditions, $c_{\mathrm{B}} + c_{\mathrm{F}}$, as shown in Equations (1) and (2). The smaller these values are, the better the attack becomes. A probabilistic extension strategically adjusts them to optimize the attack.

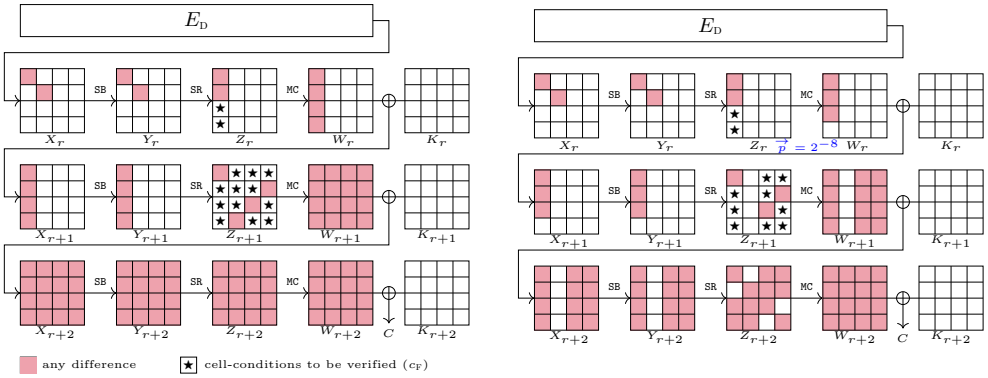
- **Impact on** $|k_{\mathrm{B}} \cup k_{\mathrm{F}}|$: The primary objective of a probabilistic extension is to reduce the number of guessed keys. By carefully selecting a low-probability trail, the diffusion of active cells is constrained, which ensures fewer active cells in the extended rounds. This in turn reduces the number of involved subkey bits. This reduction in $|k_{\mathrm{B}} \cup k_{\mathrm{F}}|$ is the main advantage of the technique.
- **Impact on** $c_{\mathrm{B}} + c_{\mathrm{F}}$: The effect of probabilistic extensions on c_{F} depends on the trade-off between the two competing factors, as given in Equation (4):

  1. The constrained diffusion reduces the ciphertext difference space, thereby reducing r_{F} and consequently c_{F}.
  2. The lower probability $\overrightarrow{p}$ increases c_{F}.

  By carefully selecting the probabilistic extension, one can reduce the value of c_{F}. The analysis is analogous for c_{B}.

In summary, the probabilistic extension technique provides a trade-off that can lead to an overall improvement in time and memory complexities, but with an increase in data complexity.

Example of probabilistic extensions. In the following, we present an example to illustrate the effectiveness of probabilistic extensions. We use AES as a toy cipher and assume independent round keys. The example is shown in Figure 6, where a 3-round extension E_{F} is appended to the ID distinguisher E_{D}, while E_{B} is omitted for simplicity (i.e., $c_{\mathrm{B}} = r_{\mathrm{B}} = 0$). We denote $eK_r = \mathtt{SR}^{-1} \circ \mathtt{MC}^{-1}(K_r)$ as the equivalent key of K_r.



Deterministic extension Probabilistic extension

Fig. 6: Toy example for comparison between deterministic extension and probabilistic extension.

For the deterministic extension example, we have $r_F = 128$. At state Z_{r+1}, 96-bit conditions need to be verified by guessing the full K_{r+2}, and at state Z_r, 16-bit conditions need to be verified by guessing $eK_{r+1}[0,1,2,3]$. Thus, $c_F = 112$ and $|k_F| = 160$.

For the probabilistic extension example, the differential propagation from Z_r to W_r occurs with probability $\overrightarrow{p} = 2^{-8}$. Since the last MC is linear, we have $r_F = 96$. First, 72-bit conditions at Z_{r+1} need to be verified by guessing 96 bits of $eK_{r+2}[0-3, 8-11, 12-15]$, and then 16-bit conditions at Z_r need to be verified by guessing $eK_{r+1}[0,1,2]$. Therefore, $c_F = 88$ and $|k_F| = 120$.

The resulting complexities are listed in Table 4. As a result, the probabilistic extension increases the data complexity by 2^8, while the time complexity of the pairs generation and guess-and-filter phrases $(D + T_1 + T_2)$ is reduced by approximately[1] 2^{44}, and the memory complexity is reduced by at least 2^{24}.

Table 4: Comparison of complexity for the examples in Figure 6.

Complexity	Deterministic extension	Probabilistic extension
$D = 2^{c_F - r_F + \mathrm{LG}(g) + n + 1}$	$2^{113+\mathrm{LG}(g)}$	$2^{121+\mathrm{LG}(g)}$
$T_1 = 2^{c_F + \mathrm{LG}(g)}$	$2^{112+\mathrm{LG}(g)}$	$2^{88+\mathrm{LG}(g)}$
$T_2 = 2^{\lvert k_F\rvert + \mathrm{LG}(g)}$	$2^{160+\mathrm{LG}(g)}$	$2^{120+\mathrm{LG}(g)}$
$M = \min\{2^{c_F+\mathrm{LG}(g)}, 2^{\lvert k_F\rvert}\}$	$\min\{2^{112+\mathrm{LG}(g)}, 2^{160}\}$	$\min\{2^{88+\mathrm{LG}(g)}, 2^{120}\}$

The probabilistic extension has been integrated into our automated tool in Section 6, and its effectiveness can be further demonstrated by our attacks on SKINNY-n-2n in Appendix L.3 and on SKINNY-n-3n in Appendix L.4.

[1] T_2 is the average time complexity of the guess-and-filter step; the exact value should be evaluated using the early-abort technique for each attack.

5 Generic Key Recovery Framework for Impossible Cryptanalysis

Inspired by recent advances in rectangle and differential attacks [20–24], this section introduces a generic key guessing strategy applicable to both ID and IB cryptanalysis. The primary advantage of this strategy is its generality: it not only unifies and encompasses all previously employed strategies for ID and IB attacks [26, 27] but, more importantly, supports arbitrary key guessing schemes.

The time and memory complexities of an ID attack are primarily determined by the number of candidate pairs, N, that must be processed in the guess-and-filter phase, as shown in Section 2.2. The core idea of our generic framework is to reduce N by performing an initial filtering step during data collection. This is achieved by pre-guessing a subset of the subkey bits, which allows many incorrect pairs to be discarded much earlier and more efficiently.

This strategy introduces a critical trade-off. The initial data processing cost increases due to the pre-guessing step. However, the subsequent guess-and-filter phase becomes more efficient. As illustrated in Figure 7, this approach generalizes previous methods by allowing an arbitrary portion of the backward subkey (k_{B}) and forward subkey (k_{F}) to be guessed before the main filtering phase. We denote these pre-guessed subkey bits as $k'_{\mathrm{B}} \subseteq k_{\mathrm{B}}$ and $k'_{\mathrm{F}} \subseteq k_{\mathrm{F}}$, all related notations are listed in Figure 7. This pre-guessing enables the verification of a subset of bit-conditions, c'_{B} and c'_{F}, during the data collection phase. Figure 8 provides a conceptual comparison between our generic approach and previous strategies.

This new strategy leads to tangible improvements, most notably a significant reduction in the memory complexity for both ID and IB attacks, and a potential reduction in time complexity, especially for IB attacks. The following subsections provide a detailed framework and complexity analysis, starting with ID attacks. The structure of our framework is inspired by the recent work in rectangle attacks [22]. The corresponding framework for IB attacks, which follows similar principles, is deferred to Appendix F for completeness.

5.1 Generic Key Recovery Framework for ID Attacks

The key recovery process for ID attacks using the arbitrary key guessing strategy can be divided into three main phases:

1. **Data Collection and Pre-filtering:** First, prepare 2^y structures of plaintexts, each of size $2^{r_{\mathrm{B}}}$, yielding a data complexity of $D = 2^{y+r_{\mathrm{B}}}$. For each of the $2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}|}$ pre-guessed subkeys, perform partial encryption and decryption on the collected data to get intermediate states (P^*, C^*). This step verifies the c'_{B}-bit conditions on the plaintext side, partitioning each initial structure into $2^{r_{\mathrm{B}}-c'_{\mathrm{B}}}$ distinct sub-structures.

   To find pairs satisfying the forward c'_{F}-bit conditions, we process each sub-structure using a hash table. The intermediate state C^* are inserted into the table, indexed by the $n-(r_{\mathrm{F}}-c'_{\mathrm{F}})$ unconstrained bits. A collision identifies a pair satisfying

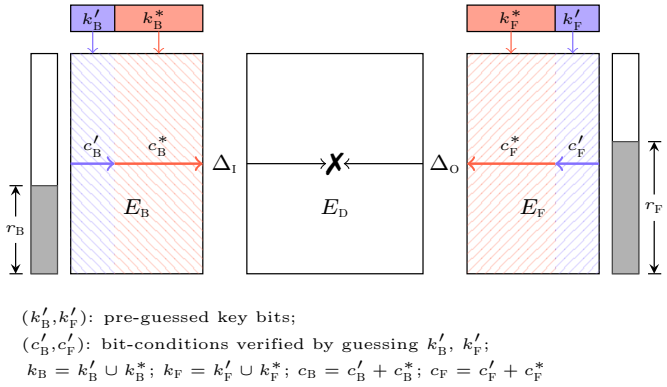


Fig. 7: The framework for generic key guessing in ID and IB cryptanalysis.

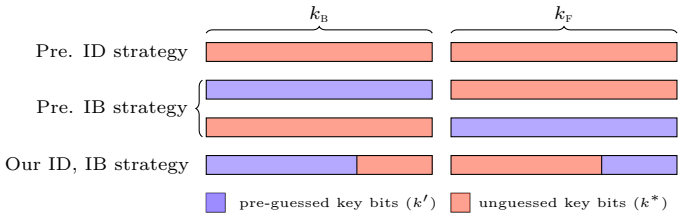


Fig. 8: The previous and our key guessing strategies in ID and IB cryptanalysis. The previous IB strategy [26, 27] is referred to Appendix D.

the forward c'_F-bit conditions. This produces a set of N candidate pairs:

$$N = (2^y \cdot 2^{c'_B}) \cdot (2^{2(r_B - c'_B)-1}) \cdot (2^{-(n-(r_F-c'_F))}) = D \cdot 2^{r_B+r_F-c'_B-c'_F-n-1}.$$

2. **Guess-and-Filter:** For each of the N candidate pairs, guess the remaining subkey bits, $k^*_B \cup k^*_F$. Any subkey guess that satisfies the impossible distinguisher over the remaining c^*_B- and c^*_F-bit conditions is discarded. A wrong key survives this filtering phase with probability $P = (1 - 2^{-(c^*_B+c^*_F)})^N$. To discard a wrong key with high probability (i.e., setting $P = 2^{-g}$), the required number of pairs N is approximately:

$$N \approx 2^{c^*_B+c^*_F+\mathrm{LG(g)}}.$$

3. **Exhaustive Search:** Finally, exhaustively search any remaining key candidates.

5.1.1 Complexity Analysis

The generic framework introduces a trade-off between the different complexity metrics. While the pre-filtering step adds to the initial computational cost, it provides a substantial reduction in memory requirements.

Memory Complexity. The primary advantage of the generic framework is the reduction in memory complexity. The memory required is determined by the number of pairs N or the size of the remaining key space to be tested in the guess-and-filter phase:

$$M = \min\{2^{c_{\mathrm{B}}^* + c_{\mathrm{F}}^* + \mathrm{LG}(g)}, 2^{|k_{\mathrm{B}}^* \cup k_{\mathrm{F}}^*|}\}$$

In the classical ID attack framework (as described in Section 2.2), the memory complexity is $\min\{2^{c_{\mathrm{B}}+c_{\mathrm{F}}+\mathrm{LG}(g)}, 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|}\}$. By pre-guessing keys, we ensure that $c_{\mathrm{B}}^* + c_{\mathrm{F}}^* < c_{\mathrm{B}} + c_{\mathrm{F}}$ and $|k_{\mathrm{B}}^* \cup k_{\mathrm{F}}^*| < |k_{\mathrm{B}} \cup k_{\mathrm{F}}|$. Since in most practical attacks the number of pairs is the bottleneck ($N < 2^{|k_{\mathrm{B}}^* \cup k_{\mathrm{F}}^*|}$), the reduction from $c_{\mathrm{B}} + c_{\mathrm{F}}$ to $c_{\mathrm{B}}^* + c_{\mathrm{F}}^*$ directly leads to a smaller memory complexity. The effectiveness of this reduction is demonstrated in our attacks on SKINNY and Midori64 in Appendices L.3, L.4, and M.2.

Time Complexity. The total time complexity of the attack is the sum of the three phases:

$$T = (D + (T_{\text{pre-filter}} + T_{\text{filter}})C_{E'} + T_{\text{search}})C_E$$

where C_E and $C_{E'}$ represent the costs of full and partial encryption, respectively. The components are:

- *Data Collection and Pre-filtering:* $D + T_{\text{pre-filter}} = D + 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}|} \cdot D$.
- *Guess-and-Filter:* This is composed of processing N pairs for each pre-guessed key (T_1) and then testing the remaining keys (T_2).

$$\begin{aligned} T_{\text{filter}} = T_1 + T_2 &= 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}|} \cdot N + 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|} \cdot \frac{N}{2^{c_{\mathrm{B}}^* + c_{\mathrm{F}}^*}} \\ &= 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| + c_{\mathrm{B}}^* + c_{\mathrm{F}}^* + \mathrm{LG}(g)} + 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}| + \mathrm{LG}(g)} \end{aligned}$$

- *Exhaustive Search:* $T_{\text{search}} = 2^{k-g}$.

The framework offers a potential reduction in the overall time complexity if the condition $|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| < c'_{\mathrm{B}} + c'_{\mathrm{F}}$ holds, as this would reduce the term T_1. However, this condition is rarely satisfied in practice, making the trade-off primarily beneficial for memory.

Data Complexity. The data complexity remains unchanged by this framework. To successfully filter out incorrect keys, the attack requires a number of pairs N that depends on the distinguisher's properties, leading to:

$$D = N \cdot 2^{n - r_{\mathrm{B}}^* - r_{\mathrm{F}}^* + 1} = 2^{n + c_{\mathrm{B}} + c_{\mathrm{F}} - r_{\mathrm{B}} - r_{\mathrm{F}} + \mathrm{LG}(g) + 1}$$

For the related-(twea)key setting, a detailed analysis is provided in Appendix E.

6 Automated Tool for Searching the Full Impossible Cryptanalysis

This section presents our new MILP-based tool for searching full ID and IB attacks. The tool is flexible and supports several operational modes: (1) a full search for

a complete attack, including the distinguisher and key recovery; (2) a standalone search for a distinguisher; and (3) an optimization of the key recovery for a given distinguisher, for both single-key and related-key settings[2]. The constraints are synthesized from state-of-the-art models [19, 22, 27, 41], with detailed inequalities listed in Appendix J. The source code is publicly available at: https://github.com/ImpossibleCryptanalysis-2024/Tool.

The model decomposes the target cipher as $E = E_{\mathrm{F}} \circ E_{\mathrm{D}} \circ E_{\mathrm{B}} = E_{\mathrm{F}} \circ E_1 \circ E_0 \circ E_{\mathrm{B}}$, and assumes a standard round function structure

$$W_{r-1} \xrightarrow{\oplus K_r} X_r \xrightarrow{\texttt{SB}} Y_r \xrightarrow{\texttt{SR}} Z_r \xrightarrow{\texttt{MC}} W_r.$$

The upper trail includes E_0 and E_{B}, while the lower trail includes E_1 and E_{F}. We use $X^{up}_{r,i}$ to denote the i-th cell in the internal state before the SB operation of the r-th round in the upper trail, and $X^{lo}_{r,i}$ to denote the i-th cell in the internal state before the SB operation of the r-th round in the lower trail. Similarly, $Y^{up}_{r,i}$, $Y^{lo}_{r,i}$, $K^{up}_{r,i}$, $K^{lo}_{r,i}$, etc. represent the various internal states and key cells.

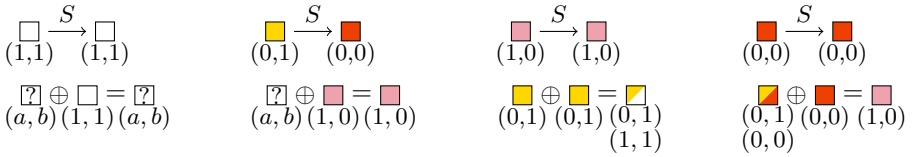
6.1 Modeling the Difference Propagation in Distinguishers

Following the miss-in-the-middle technique, the model searches for a characteristic with probability 1 over E_0 and another one over E_1^{-1} to construct an impossible distinguisher.

We use two binary variables, s_0 and s_1, to model four possible difference statuses for each state or key cell, as illustrated in the legend below.

- $(x^{s_0}, x^{s_1}) = (1,1)$ zero difference
- $(x^{s_0}, x^{s_1}) = (0,1)$ fixed nonzero difference
- $(x^{s_0}, x^{s_1}) = (1,0)$ any
- $(x^{s_0}, x^{s_1}) = (0,0)$ any but nonzero

Modeling Components. The model captures the deterministic propagation rules for each component of the round function. The rules for the S-box (SubBytes) and the XOR operation (the basis for MixColumns) are visualized below[3]. The specific linear inequalities that enforce these rules are provided in Appendix J.



Enforcing Contradictions. For ID attacks, a contradiction is a direct conflict between the output state of the upper trail and the input state of the lower trail.

[2] In this work, we have searched for distinguishers and attacks separately on all targeted ciphers, ensuring that the distinguishers in all attacks are those with the longest rounds. The distinguishers used in the attacks in Section 7 and Appendices L.3, L.4 and M.2 are the same as those proposed in previous works, which can be considered as the distinguishers that yield the optimal attacks. The distinguishers used in the optimal attacks described in Appendices L.2, N.2 and N.3 are reported for the first time.

[3] (a, b) refers to any value including $(0,0)$, $(0,1)$, $(1,0)$, and $(1,1)$.

$$\bigvee_{i=0}^{15}\left(\begin{array}{l}(X^{up,s_0}_{r_m,i}=0\wedge X^{up,s_1}_{r_m,i}=1\wedge Y^{lo,s_0}_{r_m,i}=0\wedge Y^{lo,s_1}_{r_m,i}=1)\vee\\(X^{up,s_0}_{r_m,i}=0\wedge X^{up,s_1}_{r_m,i}=0\wedge Y^{lo,s_0}_{r_m,i}=1\wedge Y^{lo,s_1}_{r_m,i}=1)\vee\\(X^{up,s_0}_{r_m,i}=1\wedge X^{up,s_1}_{r_m,i}=1\wedge Y^{lo,s_0}_{r_m,i}=0\wedge Y^{lo,s_1}_{r_m,i}=0)\end{array}\right)=1,$$

where r_m is set before running the model.

Let L^E (resp. L^D) be a binary 16×16 matrix to describe the linear layer (combination of SR and MC (resp. $\mathtt{SR}^{-1}$ and $\mathtt{MC}^{-1}$)), $L^E(i)$ (resp. $L^D(i)$) be the set of the indexes j such that the coefficient $L^E_{i,j}=1$ (resp. $L^D_{i,j}=1$) in the matrix L^E (resp. L^D). We provide the matrices L^E and L^D for SKINNY and AES in Appendix J.6. Additionally, we denote ${}^tL^E$ (resp. ${}^tL^D$) as the transpose of L^E (resp. L^D).

For IB attacks, we use constraints derived from our new tables to model contradictions over one, two, or three rounds:

$$\mathtt{contr}_1=\bigvee_{i=0}^{15}(X^{up,s_0}_{r_m,i}=0\wedge X^{up,s_1}_{r_m,i}=1\wedge Y^{lo,s_0}_{r_m,i}=0\wedge Y^{lo,s_1}_{r_m,i}=1)$$

for describing contradiction through single-round E_m ($\mathtt{BCT}=0$),

$$\mathtt{contr}_{2\mathtt{U}}=\bigvee_{i=0}^{15}\left(\begin{array}{l}X^{up,s_0}_{r_m,i}=0\wedge X^{up,s_1}_{r_m,i}=1\wedge\\\left(\bigvee_{j\in L^E(i)}Y^{lo,s_0}_{r_m+1,j}=0\wedge Y^{lo,s_1}_{r_m+1,j}=1\wedge\left(\bigwedge_{k\in {}^tL^E(j)\setminus i}X^{up,s_0}_{r_m,k}=1\wedge X^{up,s_1}_{r_m,k}=1\right)\right)\end{array}\right)$$

and

$$\mathtt{contr}_{2\mathtt{L}}=\bigvee_{i=0}^{15}\left(\begin{array}{l}Y^{lo,s_0}_{r_m+1,i}=0\wedge Y^{lo,s_1}_{r_m+1,i}=1\wedge\\\left(\bigvee_{j\in L^D(i)}X^{up,s_0}_{r_m,j}=0\wedge X^{up,s_1}_{r_m,j}=1\wedge\left(\bigwedge_{k\in {}^tL^D(j)\setminus i}Y^{lo,s_0}_{r_m+1,k}=1\wedge Y^{lo,s_1}_{r_m+1,k}=1\right)\right)\end{array}\right)$$

for describing contradictions through two-round E_m ($\mathtt{contr}_{2\mathtt{U}}$ for $\mathtt{iUBCT}=0$, $\mathtt{contr}_{2\mathtt{L}}$ for $\mathtt{iLBCT}=0$), then set $\mathtt{contr}_2=\mathtt{contr}_{2\mathtt{U}}\wedge\mathtt{contr}_{2\mathtt{L}}$. Figure 9 provides a toy example with AES-like linear layer of $\mathtt{contr}_{2\mathtt{U}}$ when $i=0, j\in\{0,1,2,3\}, k\in\{5,10,15\}$, and $\mathtt{contr}_{2\mathtt{L}}$ when $i=0, j\in\{0,5,10,15\}, k\in\{1,2,3\}$. Similarly, we can set constraints

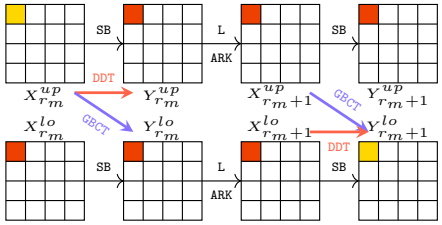


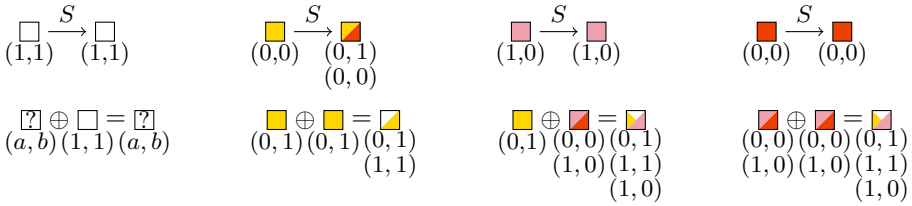
Fig. 9: A toy example of the contradiction $\mathtt{contr}_2$ for two-round E_m.

for describing contradiction through three-round E_m ($\mathtt{i3UBCT}/\mathtt{i3MBCT}/\mathtt{i3LBCT}=0$). The model then requires that at least one such contradiction exists ($\bigvee\mathtt{contr}_\mathtt{i}=1$).

6.2 Modeling the Generic Key Recovery

The model's second major function is to find the optimal key recovery for E_{B} and E_{F}.

Probabilistic Extensions. The model can explore probabilistic key recovery extensions by using a set of relaxed MILP constraints for the propagation through `SubBytes` and `MixColumns`, allowing for probabilistic transitions, as illustrated below. The specific linear inequalities that enforce these rules are provided in Appendix J.



Identifying bit conditions $c_{\mathrm{B}}, c_{\mathrm{F}}$. The bit conditions to be verified in E_{B} will only exist in states Y and W, while the cell conditions that need to be verified in E_{F} will only exist in states X and Z. Taking E_{B} as an example, the conditions in Y can be identified by

$$Y_{r,i}^{up,c} = \neg Y_{r,i}^{up,s_0} \wedge Y_{r,i}^{up,s_1},$$

and the conditions in W can be identified by

$$W_{r,i}^{up,c} = W_{r,i}^{up,s_1} \wedge \Big(\bigvee_{j \in {}^{t}L^{D}(i)} \neg X_{r,j}^{up,s_1} \Big),$$

thus we can obtain $c_{\mathrm{B}} = \sum_r \sum_i (Y_{r,i}^{up,c} + W_{r,i}^{up,c})$ for the complexity calculations. The constraints in E_{F} are similar.

Identifying Key Recovery Parameters. To enable the generic key-guessing strategy, we introduce dedicated binary variables to track all relevant parameters. For each key cell $K_{r,i}$, we use $K_{r,i}^{g}$ to denote if the cell is involved in the attack ($k_{\mathrm{B}} \cup k_{\mathrm{F}}$) and $K_{r,i}^{p}$ to denote if it is guessed in the pre-filtering phase ($k'_{\mathrm{B}} \cup k'_{\mathrm{F}}$). There is a clear constraint

$$\texttt{if } K_{r,i}^{p} = 1, \texttt{ then } K_{r,i}^{g} = 1,$$

meaning if a key cell is pre-guessed, it must be involved in the attack.

Take E_{B} as an example. The subkey cells involved in verifying the conditions of Y_r satisfy

$$\texttt{if } Y_{r,i}^{up,c} = 1, \texttt{ then } K_{r,i}^{up,g} = 1.$$

The subkey cells involved in verifying the conditions of W_r can be computed as

$$\texttt{if } \big(W_{r,i}^{up,c} = 1 \wedge (X_{r,j}^{up,s_0} + X_{r,j}^{up,s_1} \leq 1)\big), \texttt{ then } K_{r,j}^{up,g} = 1 \quad \texttt{for all } j \in {}^{t}L^{D}(i).$$

Besides, we can backtrack to compute all the involved subkey cells from round 0 using

$$\texttt{if } K_{r,i}^{up,g} = 1, \texttt{ then } K_{r-1,j}^{up,g} = 1 \texttt{ for } \forall j \in {}^{t}L^{D}(i).$$

Thus, we obtain $k_{\mathrm{B}} = \sum_r \sum_i K^{up,g}_{r,i}$.

The model also introduces two variables Y^v_r and W^v_r to indicate that the bit conditions are verified under the pre-guessed key K^p_r. The corresponding constraints, similar to those described above, are provided in Appendix J.5. Analogous constraints hold for E_{F}.

Finally, we obtain r'_{B}, r'_{F}, c'_{B}, c'_{F}, k'_{B}, and k'_{F}. In addition, the model also supports key-bridging techniques to reduce the total number of guessed keys, following the approaches in [13, 22, 42].

6.3 Objective Function and Performance

Objective function. The primary goal is to find the attack with the lowest complexity. The objective function is set to minimize a heuristic for the total time complexity T, which balances the time complexities of all phases. Since all parameters are known, the model can also be configured to minimize data or memory complexity directly.

Performance and Verification. In practice, the model remains manageable for modern ciphers. For instance, when searching for a full 19-round attack on SKINNY-n-n, our MILP model consists of approximately 20986 variables and 18722 constraints. Using the Gurobi solver on a standard workstation, the search for the optimal attack completes in approximately 0.25 hours. All other experimental results in this work are listed in Table 5.

Table 5: Experiments on target ciphers. $\#V$ denotes the number of variables. $\#C$ denotes the number of constraints. T denotes the model solving runtime.

Environment: Intel Xeon Gold 5220R CPU @ 2.20GHz, Gurobi 10.01						
	SKINNY-n-2n	SKINNY-n-3n	Midori64	SKINNYee	Deoxys-BC-256	Deoxys-BC-384
$\#V$	21018	23210	16090	32657	11766	16326
$\#C$	18769	20816	36400	29781	7617	10801
T	59 mins	35 mins	4.4 hrs	11 secs	1 sec	35 secs

The final output of the tool is a truncated characteristic, which is then instantiated and verified as outlined in Algorithm 3.

7 Applications

This section demonstrates the practical impact of the techniques developed in this paper. We apply our methods to SKINNY, SKINNYee, Midori, and Deoxys-BC. Due to page limits, we present the detailed attacks on SKINNYee in this section. The remaining attacks on SKINNY, Midori64, and Deoxys-BC are deferred to the Appendices L.2, L.3, L.4, M.2, N.2 and N.3.

7.1 30-Round Impossible Boomerang Attack on SKINNYee

We propose a 30-round related-tweak IB attack against SKINNYee, which is obtained by our automated tool. The specification of SKINNYee is provided in Appendix K. In this attack, we use a 21-round related-tweak IB distinguisher, which is the same distinguisher used in [26]:

$$\begin{aligned}(\Delta Y_5, \Delta ST_5) &= ((0000|0a00|0000|0000), (0000|0a00)) \\ &\nrightarrow \\ (\nabla Z_{26}, \nabla ST_{26}) &= ((0000|000b|0000|0000), (0000|000b)),\end{aligned}$$

where Δ denotes the difference in the upper characteristic and ∇ denotes the difference in the lower characteristic of the boomerang trail.

We prefix 6 rounds at the beginning and append 3 rounds at the end of the distinguisher to mount the attack, as shown in Figure 10. From the figure, we can get the parameters used for this attack: $r_{\mathrm{B}} = 12c$, $c_{\mathrm{B}} = 12c$, $r_{\mathrm{F}} = 9c$, $c_{\mathrm{F}} = 9c$, $|k_{\mathrm{B}} \cup k_{\mathrm{F}}| = 26c$, $c'_{\mathrm{B}} = 5c$, $c'_{\mathrm{F}} = 9c$, $|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| = 15c$, where $c = 4$ denotes the cell size. In the key-recovery extensions of this attack, it adopts deterministic extensions.

Data Collection. This phase follows a "ciphertext-first" approach. We pre-guess 2^{15c} possible values of $K_0[0-7]$, $K_1[0,3,4,5,6,7]$ and $K_3[0]$ to generate two sets as:

$$\begin{aligned}S_1 &= \{(P_1^*, C_1^*, P_3^*, C_3^*) \mid C_1^* \text{ and } C_3^* \text{ are colliding in } n - r_{\mathrm{F}}^* = 16c \text{ bits}\}, \\ S_2 &= \{(P_2^*, C_2^*, P_4^*, C_4^*) \mid C_2^* \text{ and } C_4^* \text{ are colliding in } n - r_{\mathrm{F}}^* = 16c \text{ bits}\}.\end{aligned}$$

According to Appendix F, the size of each set is $(D/4)^2 \cdot 2^{r_{\mathrm{F}}^* - n} = 2^{n + c_{\mathrm{F}}^* + \mathrm{LG}(g)} = 2^{16c + \mathrm{LG}(g)}$. From the two sets, we can construct $Q = 2^{2c_{\mathrm{B}}^* + 2c_{\mathrm{F}}^* + \mathrm{LG}(g)} = 2^{14c + \mathrm{LG}(g)}$ quartets for performing guess-and-filter phase.

Guess-and-Filter. For Q quartets under each pre-guessed subkey bits:

1. *Satisfying the cell conditions in* ΔX_3. Guess $K_1[1]$ and we can compute $\Delta W_2[3]$. The condition $\Delta W_2[3] = \Delta W_2[11]$ will lead to two c-bit filters on both sides of the boomerang. The time complexity of this step is $2^{16c} \cdot Q \cdot 4 \cdot \frac{1}{30 \cdot 16} = Q2^{16c - 6.91}$.
2. *Satisfying the cell conditions in* ΔX_3. Guess $K_1[2]$ and we can compute $\Delta W_2[10]$. The condition $\Delta W_2[2] \oplus \Delta W_2[10] \oplus \Delta W_2[14] = 0$ will lead to two c-bit filters on both sides of the boomerang. The time complexity of this step is $2^{17c} \cdot Q \cdot 2^{-2c} \cdot 4 \cdot \frac{1}{30 \cdot 16} = Q2^{15c - 6.91}$.
3. *Satisfying the cell conditions in* ΔX_4. Guess $K_2[0]$ and compute $\Delta W_3[8]$. The condition $\Delta W_3[4] = \Delta W_3[8]$ will lead to two c-bit filters. Guess $K_2[2,5]$ and compute $\Delta W_3[0]$. The condition $\Delta W_3[0] = \Delta W_3[8]$ will also lead to two c-bit filters. The time complexity of this step is $2^{18c} \cdot Q \cdot 2^{-4c} \cdot 4 \cdot \frac{1}{30 \cdot 16} + 2^{20c} \cdot Q \cdot 2^{-6c} \cdot 4 \cdot \frac{2}{30 \cdot 16} = Q2^{14c - 5.32}$.
4. *Satisfying the cell conditions in* ΔX_5. Guess $K_2[3]$ and $K_3[1]$. The condition $\Delta W_4[5] = \Delta W_4[9]$ will lead to two c-bit filters. The time complexity of this step is $2^{22c} \cdot Q \cdot 2^{-8c} \cdot 4 \cdot \frac{2}{30 \cdot 16} = Q2^{14c - 5.91}$.

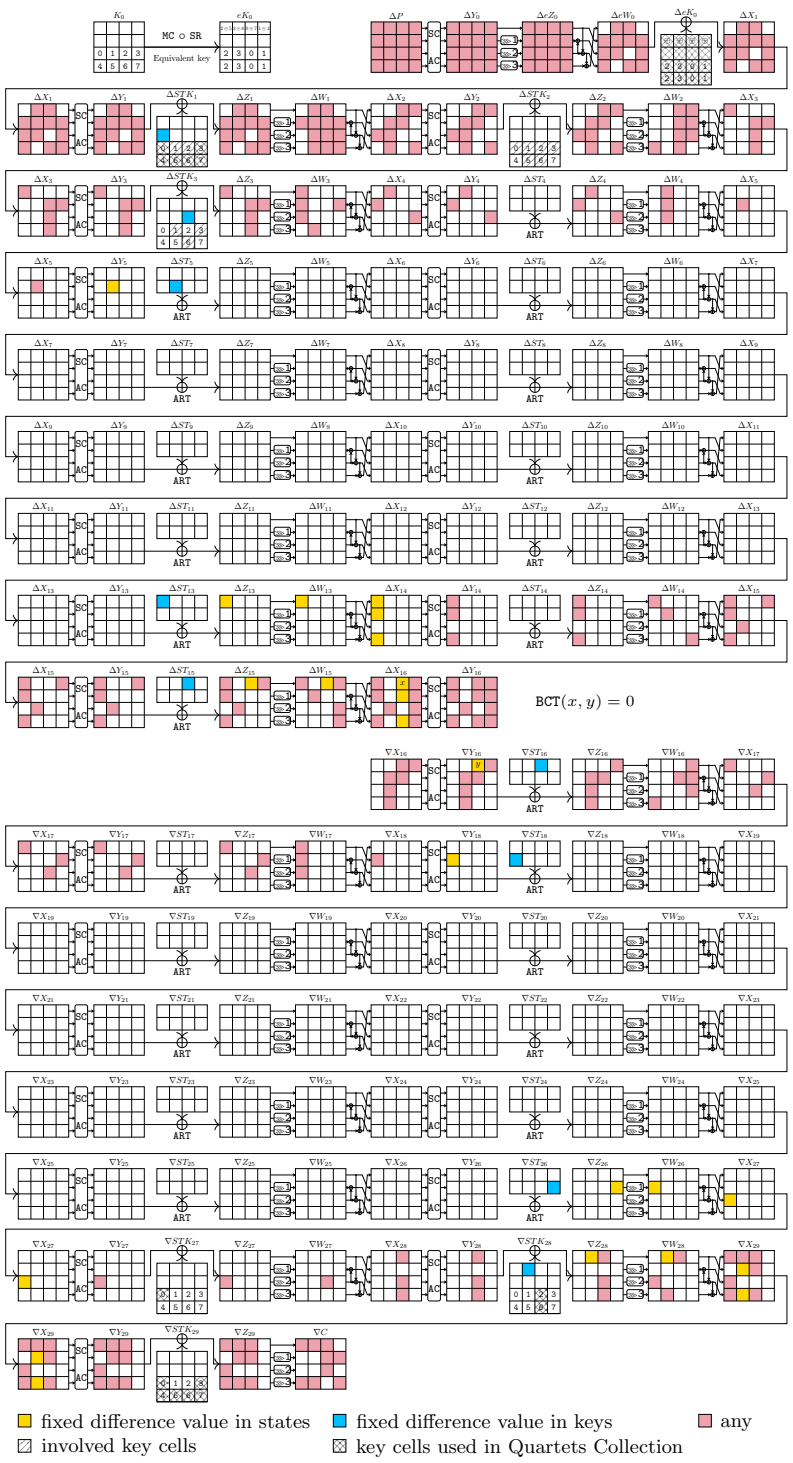


Fig. 10: The related-tweak impossible boomerang attack against 30-round SKINNYee.

5. *Satisfying the cell conditions in* ΔX_5 *and* ΔY_5. Guess $K_2[1,6]$ and $K_3[3,6]$. The conditions with $\Delta W_4[1] = \Delta W_4[9]$ and the known difference value of $Y_5[5]$ will lead to four c-bit filters. The time complexity of this step is $2^{26c} \cdot Q \cdot 2^{-10c} \cdot 4 \cdot \frac{4}{30 \cdot 16} = Q2^{16c-4.91}$.

Complexity. The data complexity is $D = 2^{16c+\frac{\mathrm{LG}(g)}{2}+2}$. The time complexity primarily consists of partial encryption/decryption, generating sets in quartets collection, guess-and-filter and exhaustive search, represented as $2^{31c+\frac{\mathrm{LG}(g)}{2}+2} \cdot \frac{15}{30 \cdot 16} + 2^{31c+\mathrm{LG}(g)+1} \cdot \frac{15}{30 \cdot 16} + 2^{30c+\mathrm{LG}(g)-5.12} + 2^{32c-g}$. In order to achieve the optimal trade-off between the time and data complexities, we set $g = 6$. Finally, we have $D = 2^{67.03}$, $T = 2^{123.61}, M = 2^{44}$.

8 Conclusion

This paper systematically addressed several long-standing limitations in impossible cryptanalysis by introducing new techniques for both distinguisher construction and key recovery optimization. For IB attacks, we introduced the `iUBCT` and `iLBCT`: lightweight boomerang tables that overcome the computational and modeling bottlenecks of existing methods for finding multi-round contradictions. Their efficacy was demonstrated by the discovery of a new 19-round IB distinguisher for SKINNY-128-384, extending the previous state of the art by one round. For the key recovery phase, we modernized the attack process by adapting and formalizing two powerful techniques from related fields. We provided the first rigorous analysis of probabilistic extensions in this context and integrated a generic key guessing framework that enables a systematic search for attacks with optimal complexities, particularly for memory.

These theoretical advancements were unified within a powerful MILP-based tool, which led to a series of the best-known impossible cryptanalysis attacks on ciphers such as SKINNY, Midori, and Deoxys-BC, including the first 30-round attack on SKINNYee. While our automated framework currently targets SPN ciphers, this work opens up several avenues for future research. Extending our tool to support other structures like Feistel networks or ARX designs is an immediate direction. Furthermore, a deeper theoretical study of the interaction between probabilistic extensions and our generic key-guessing framework could reveal new optimal trade-offs. Finally, a key open problem is to develop tools that combine the efficiency of our `iUBCT`/`iLBCT` with the full coverage of the `DBCT`*.

Acknowledgements. This work was supported by the National Natural Science Foundation of China (Grant No. 62302293).

Declarations

The authors declare that they have no competing interests.

References

[1] Knudsen, L.: DEAL-a 128-bit block cipher. complexity **258**(2), 216 (1998)

[2] Biham, E., Biryukov, A., Shamir, A.: Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials. In: Stern, J. (ed.) Advances in Cryptology – EUROCRYPT'99. Lecture Notes in Computer Science, vol. 1592, pp. 12–23. Springer, Prague, Czech Republic (1999). https://doi.org/10.1007/3-540-48910-X_2

[3] Lu, J.: Cryptanalysis of block ciphers. PhD thesis, University of London UK (2008)

[4] Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. In: Menezes, A.J., Vanstone, S.A. (eds.) Advances in Cryptology – CRYPTO'90. Lecture Notes in Computer Science, vol. 537, pp. 2–21. Springer, Santa Barbara, CA, USA (1991). https://doi.org/10.1007/3-540-38424-3_1

[5] Biham, E., Biryukov, A., Shamir, A.: Miss in the middle attacks on IDEA and Khufu. In: Knudsen, L.R. (ed.) Fast Software Encryption – FSE'99. Lecture Notes in Computer Science, vol. 1636, pp. 124–138. Springer, Rome, Italy (1999). https://doi.org/10.1007/3-540-48519-8_10

[6] Cui, T., Jia, K., Fu, K., Chen, S., Wang, M.: New Automatic Search Tool for Impossible Differentials and Zero-Correlation Linear Approximations. Cryptology ePrint Archive, Report 2016/689. https://eprint.iacr.org/2016/689 (2016)

[7] Sasaki, Y., Todo, Y.: New impossible differential search tool from design and cryptanalysis aspects - revealing structural properties of several ciphers. In: Coron, J.-S., Nielsen, J.B. (eds.) Advances in Cryptology – EUROCRYPT 2017, Part III. Lecture Notes in Computer Science, vol. 10212, pp. 185–215. Springer, Paris, France (2017). https://doi.org/10.1007/978-3-319-56617-7_7

[8] Sun, L., Gerault, D., Wang, W., Wang, M.: On the usage of deterministic (RK) TDs and MDLAs for SPN ciphers. IACR Transactions on Symmetric Cryptology **2020**(3), 262–287 (2020) https://doi.org/10.13154/tosc.v2020.i3.262-287

[9] Lu, J., Dunkelman, O., Keller, N., Kim, J.: New impossible differential attacks on AES. In: Progress in Cryptology - INDOCRYPT 2008, 9th International Conference on Cryptology in India, Kharagpur, India, December 14-17, 2008. Proceedings. Lecture Notes in Computer Science, vol. 5365, pp. 279–293. https://doi.org/10.1007/978-3-540-89754-5_22

[10] Boura, C., Lallemand, V., Naya-Plasencia, M., Suder, V.: Making the impossible possible. Journal of Cryptology **31**(1), 101–133 (2018) https://doi.org/10.1007/s00145-016-9251-7

[11] Boura, C., Naya-Plasencia, M., Suder, V.: Scrutinizing and improving impossible differential attacks: Applications to CLEFIA, Camellia, LBlock and Simon. In: Sarkar, P., Iwata, T. (eds.) Advances in Cryptology – ASIACRYPT 2014, Part I. Lecture Notes in Computer Science, vol. 8873, pp. 179–199. Springer, Kaoshiung, Taiwan, R.O.C. (2014). https://doi.org/10.1007/978-3-662-45611-8_10

[12] Derbez, P., Fouque, P.-A.: Automatic search of meet-in-the-middle and impossible differential attacks. In: Robshaw, M., Katz, J. (eds.) Advances in Cryptology – CRYPTO 2016, Part II. Lecture Notes in Computer Science, vol. 9815, pp. 157–184. Springer, Santa Barbara, CA, USA (2016). https://doi.org/10.1007/978-3-662-53008-5_6

[13] Hadipour, H., Sadeghi, S., Eichlseder, M.: Finding the impossible: Automated search for full impossible-differential, zero-correlation, and integral attacks. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology – EUROCRYPT 2023, Part IV. Lecture Notes in Computer Science, vol. 14007, pp. 128–157. Springer, Lyon, France (2023). https://doi.org/10.1007/978-3-031-30634-1_5

[14] Hadipour, H., Gerhalter, S., Sadeghi, S., Eichlseder, M.: Improved search for integral, impossible differential and zero-correlation attacks application to Ascon, ForkSKINNY, SKINNY, MANTIS, PRESENT and QARMAv2. IACR Transactions on Symmetric Cryptology **2024**(1), 234–325 (2024) https://doi.org/10.46586/tosc.v2024.i1.234-325

[15] Song, L., Fu, Q., Yang, Q., Lv, Y., Hu, L.: Generalized Impossible Differential Attacks on Block Ciphers: Application to SKINNY and ForkSKINNY. Cryptology ePrint Archive, Paper 2024/1908 (2024). https://eprint.iacr.org/2024/1908

[16] Phan, R.C.: Impossible differential cryptanalysis of 7-round advanced encryption standard (AES). Inf. Process. Lett. **91**(1), 33–38 (2004) https://doi.org/10.1016/J.IPL.2004.02.018

[17] Bahrak, B., Aref, M.R.: A novel impossible differential cryptanalysis of AES. In: Proceedings of the Western European Workshop on Research in Cryptology, vol. 2007 (2007). Citeseer

[18] Mala, H., Dakhilalian, M., Rijmen, V., Modarres-Hashemi, M.: Improved impossible differential cryptanalysis of 7-round AES-128. In: Progress in Cryptology - INDOCRYPT 2010 - 11th International Conference on Cryptology in India, Hyderabad, India, December 12-15, 2010. Proceedings. Lecture Notes in Computer Science, vol. 6498, pp. 282–291. Springer, Hyderabad, India (2010). https://doi.org/10.1007/978-3-642-17401-8_20 . https://doi.org/10.1007/978-3-642-17401-8_20

[19] Song, L., Yang, Q., Chen, Y., Hu, L., Weng, J.: Probabilistic extensions: A one-step framework for finding rectangle attacks and beyond. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024, Part I. Lecture Notes in

Computer Science, vol. 14651, pp. 339–367. Springer, Zurich, Switzerland (2024). https://doi.org/10.1007/978-3-031-58716-0_12

[20] Zhao, B., Dong, X., Meier, W., Jia, K., Wang, G.: Generalized related-key rectangle attacks on block ciphers with linear key schedule: applications to SKINNY and GIFT. Designs, Codes and Cryptography **88**(6), 1103–1126 (2020) https://doi.org/10.1007/s10623-020-00730-1

[21] Dong, X., Qin, L., Sun, S., Wang, X.: Key guessing strategies for linear key-schedule algorithms in rectangle attacks. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022, Part III. Lecture Notes in Computer Science, vol. 13277, pp. 3–33. Springer, Trondheim, Norway (2022). https://doi.org/10.1007/978-3-031-07082-2_1

[22] Song, L., Zhang, N., Yang, Q., Shi, D., Zhao, J., Hu, L., Weng, J.: Optimizing rectangle attacks: A unified and generic framework for key recovery. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology – ASIACRYPT 2022, Part I. Lecture Notes in Computer Science, vol. 13791, pp. 410–440. Springer, Taipei, Taiwan (2022). https://doi.org/10.1007/978-3-031-22963-3_14

[23] Song, L., Liu, H., Yang, Q., Chen, Y., Hu, L., Weng, J.: Generic differential key recovery attacks and beyond. In: Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. Lecture Notes in Computer Science, vol. 15490, pp. 361–391. Springer, ??? (2024). https://doi.org/10.1007/978-981-96-0941-3_12 . https://doi.org/10.1007/978-981-96-0941-3_12

[24] Yang, Q., Song, L., Zhang, N., Shi, D., Wang, L., Zhao, J., Hu, L., Weng, J.: Optimizing rectangle and boomerang attacks: A unified and generic framework for key recovery. J. Cryptol. **37**(2), 19 (2024) https://doi.org/10.1007/S00145-024-09499-1

[25] Lu, J.: The (related-key) impossible boomerang attack and its application to the AES block cipher. Des. Codes Cryptogr. **60**(2), 123–143 (2011) https://doi.org/10.1007/s10623-010-9421-9

[26] Bonnetain, X., Cordero, M., Lallemand, V., Minier, M., Naya-Plasencia, M.: On impossible boomerang attacks application to Simon and SKINNYee. IACR Trans. Symmetric Cryptol. **2024**(2), 222–253 (2024) https://doi.org/10.46586/TOSC.V2024.I2.222-253

[27] Zhang, J., Wang, H., Tang, D.: Impossible boomerang attacks revisited applications to Deoxys-BC, Joltik-BC and SKINNY. IACR Trans. Symmetric Cryptol. **2024**(2), 254–295 (2024) https://doi.org/10.46586/TOSC.V2024.I2.254-295

[28] Hu, X., Feng, D., Jiao, L., Hao, Y., Gong, X., Li, Y.: A deep study of the impossible boomerang distinguishers: New construction theory and automatic search methods. IACR Cryptol. ePrint Arch., 1008 (2024)

[29] Chen, Y., Fu, Q., Zhao, N., Zhao, J., Song, L., Yang, Q.: A Holistic Framework for Impossible Boomerang Attacks. Cryptology ePrint Archive, Paper 2025/159 (2025). https://eprint.iacr.org/2025/159

[30] Bonnetain, X., Lallemand, V.: A note on the use of the double boomerang connectivity table (DBCT) for spotting impossibilities. IACR Cryptol. ePrint Arch., 1218 (2024)

[31] Liu, G., Ghosh, M., Song, L.: Security analysis of SKINNY under related-tweakey settings (long paper). IACR Transactions on Symmetric Cryptology **2017**(3), 37–72 (2017) https://doi.org/10.13154/tosc.v2017.i3.37-72

[32] Liu, Y., Xiang, Z., Chen, S., Zhang, S., Zeng, X.: A novel automatic technique based on MILP to search for impossible differentials. In: Tibouchi, M., Wang, X. (eds.) ACNS 23: 21st International Conference on Applied Cryptography and Network Security, Part I. Lecture Notes in Computer Science, vol. 13905, pp. 119–148. Springer, Kyoto, Japan (2023). https://doi.org/10.1007/978-3-031-33488-7_5

[33] Lin, L., Wu, W.: Meet-in-the-middle attacks on reduced-round Midori64. IACR Transactions on Symmetric Cryptology **2017**(1), 215–239 (2017) https://doi.org/10.13154/tosc.v2017.i1.215-239

[34] Lu, J., Kim, J., Keller, N., Dunkelman, O.: Improving the efficiency of impossible differential cryptanalysis of reduced Camellia and MISTY1. In: Malkin, T. (ed.) Topics in Cryptology – CT-RSA 2008. Lecture Notes in Computer Science, vol. 4964, pp. 370–386. Springer, San Francisco, CA, USA (2008). https://doi.org/10.1007/978-3-540-79263-5_24

[35] Cid, C., Huang, T., Peyrin, T., Sasaki, Y., Song, L.: Boomerang connectivity table: A new cryptanalysis tool. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018, Part II. Lecture Notes in Computer Science, vol. 10821, pp. 683–714. Springer, Tel Aviv, Israel (2018). https://doi.org/10.1007/978-3-319-78375-8_22

[36] Hadipour, H., Bagheri, N., Song, L.: Improved rectangle attacks on SKINNY and CRAFT. IACR Transactions on Symmetric Cryptology **2021**(2), 140–198 (2021) https://doi.org/10.46586/tosc.v2021.i2.140-198

[37] Yang, Q., Song, L., Sun, S., Shi, D., Hu, L.: New properties of the double boomerang connectivity table. IACR Transactions on Symmetric Cryptology **2022**(4), 208–242 (2022) https://doi.org/10.46586/tosc.v2022.i4.208-242

[38] Wang, L., Song, L., Wu, B., Rahman, M., Isobe, T.: Revisiting the boomerang attack from a perspective of 3-differential. IEEE Trans. Inf. Theory **70**(7), 5343–5357 (2024) https://doi.org/10.1109/TIT.2023.3324738

[39] Li, C., Wu, B., Lin, D.: Generalized boomerang connectivity table and improved cryptanalysis of GIFT. In: Inscrypt 2022, Beijing, China, December 11-13, 2022. Lecture Notes in Computer Science, vol. 13837, pp. 213–233. Springer, ??? (2022). https://doi.org/10.1007/978-3-031-26553-2_11 . https://doi.org/10.1007/978-3-031-26553-2_11

[40] Beierle, C., Jean, J., Kölbl, S., Leander, G., Moradi, A., Peyrin, T., Sasaki, Y., Sasdrich, P., Sim, S.M.: The SKINNY family of block ciphers and its low-latency variant MANTIS. In: Robshaw, M., Katz, J. (eds.) Advances in Cryptology – CRYPTO 2016, Part II. Lecture Notes in Computer Science, vol. 9815, pp. 123–153. Springer, Santa Barbara, CA, USA (2016). https://doi.org/10.1007/978-3-662-53008-5_5

[41] Derbez, P., Euler, M., Fouque, P.-A., Nguyen, P.H.: Revisiting related-key boomerang attacks on AES using computer-aided tool. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology – ASIACRYPT 2022, Part III. Lecture Notes in Computer Science, vol. 13793, pp. 68–88. Springer, Taipei, Taiwan (2022). https://doi.org/10.1007/978-3-031-22969-5_3

[42] Dunkelman, O., Keller, N., Shamir, A.: Improved single-key attacks on 8-round AES-192 and AES-256. In: Abe, M. (ed.) Advances in Cryptology – ASIACRYPT 2010. Lecture Notes in Computer Science, vol. 6477, pp. 158–176. Springer, Singapore (2010). https://doi.org/10.1007/978-3-642-17373-8_10

[43] Wang, H., Peyrin, T.: Boomerang switch in multiple rounds. IACR Transactions on Symmetric Cryptology **2019**(1), 142–169 (2019) https://doi.org/10.13154/tosc.v2019.i1.142-169

[44] Delaune, S., Derbez, P., Vavrille, M.: Catching the fastest boomerangs application to SKINNY. IACR Transactions on Symmetric Cryptology **2020**(4), 104–129 (2020) https://doi.org/10.46586/tosc.v2020.i4.104-129

[45] Dunkelman, O.: Efficient Construction of the Boomerang Connection Table. Cryptology ePrint Archive, Paper 2018/631 (2018). https://eprint.iacr.org/2018/631

[46] Naito, Y., Sasaki, Y., Sugawara, T.: Secret can be public: Low-memory AEAD mode for high-order masking. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part III. Lecture Notes in Computer Science, vol. 13509, pp. 315–345. Springer, Santa Barbara, CA, USA (2022). https://doi.org/10.1007/978-3-031-15982-4_11

[47] Naito, Y., Sasaki, Y., Sugawara, T.: Lightweight authenticated encryption mode

suitable for threshold implementation. In: Canteaut, A., Ishai, Y. (eds.) Advances in Cryptology – EUROCRYPT 2020, Part II. Lecture Notes in Computer Science, vol. 12106, pp. 705–735. Springer, Zagreb, Croatia (2020). https://doi.org/10.1007/978-3-030-45724-2_24

[48] Banik, S., Bogdanov, A., Isobe, T., Shibutani, K., Hiwatari, H., Akishita, T., Regazzoni, F.: Midori: A block cipher for low energy. In: Iwata, T., Cheon, J.H. (eds.) Advances in Cryptology – ASIACRYPT 2015, Part II. Lecture Notes in Computer Science, vol. 9453, pp. 411–436. Springer, Auckland, New Zealand (2015). https://doi.org/10.1007/978-3-662-48800-3_17

[49] Jean, J., Nikolic, I., Peyrin, T., Seurin, Y.: Deoxys v1. 41. Submitted to CAESAR **124** (2016)

[50] Jean, J., Nikolic, I., Peyrin, T.: Tweaks and keys for block ciphers: The TWEAKEY framework. In: Sarkar, P., Iwata, T. (eds.) Advances in Cryptology – ASIACRYPT 2014, Part II. Lecture Notes in Computer Science, vol. 8874, pp. 274–288. Springer, Kaoshiung, Taiwan, R.O.C. (2014). https://doi.org/10.1007/978-3-662-45608-8_15

A The Boomerang Tables

Definition 7 (UBCT, LBCT [43, 44])**.** Let S be a bijective function over $\mathbb{F}_2^n$, and $\alpha_0, \alpha_1, \beta_0, \beta_1 \in \mathbb{F}_2^n$. The Upper BCT (UBCT) and the Lower BCT (LBCT) of S are three-dimensional tables defined as:

$$\texttt{UBCT}(\alpha_0, \alpha_1, \beta_1) = \#\left\{ x \in \mathbb{F}_2^n \middle| \begin{array}{l} S(x) \oplus S(x \oplus \alpha_0) = \alpha_1 \\ S^{-1}(S(x) \oplus \beta_1) \oplus S^{-1}(S(x \oplus \alpha_0) \oplus \beta_1) = \alpha_0 \end{array} \right\},$$

$$\texttt{LBCT}(\alpha_0, \beta_0, \beta_1) = \#\left\{ x \in \mathbb{F}_2^n \middle| \begin{array}{l} S(x) \oplus S(x \oplus \beta_0) = \beta_1 \\ S^{-1}(S(x) \oplus \beta_1) \oplus S^{-1}(S(x \oplus \alpha_0) \oplus \beta_1) = \alpha_0 \end{array} \right\}.$$

B iUBCT and iLBCT of the SKINNY-64 S-box

The two tables are listed in Tables 6 and 7.

Table 6: iUBCT of SKINNY-64's S-box

$\alpha_0 \setminus \beta_2$	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	4	6	2	6	6	6	6	4	9	9	9	9	9	9	9	9
2	4	5	2	5	5	4	5	6	8	7	4	9	8	7	12	7
3	8	26	6	26	28	14	26	20	30	30	34	34	28	28	34	34
4	6	12	6	12	14	10	12	18	17	17	17	17	14	14	18	18
5	6	12	6	12	14	10	12	18	16	16	16	16	11	11	19	19
6	8	24	7	24	22	16	22	20	31	29	31	29	26	30	26	30
7	8	22	7	22	22	14	24	20	25	29	29	25	28	30	30	28
8	6	12	8	12	16	18	12	36	15	15	15	15	19	19	17	17
9	6	12	8	12	16	18	12	36	15	15	15	15	19	19	17	17
a	6	15	3	15	15	8	15	8	21	22	17	18	18	21	20	15
b	6	12	5	12	14	10	12	18	15	15	13	13	18	18	16	16
c	6	14	8	14	18	10	18	16	20	20	20	20	17	17	17	17
d	6	14	8	14	18	10	18	16	20	20	20	20	17	17	17	17
e	8	24	7	24	22	14	22	20	30	28	30	28	25	29	25	29
f	8	22	7	22	22	16	24	20	24	32	32	24	33	27	27	33

C Proof of Properties 1, 2

Proof of Property 1. In the construction of $\texttt{iUBCT}(0, \beta)$, the only solution satisfying $\texttt{DDT}(0, \alpha_1) > 0, \texttt{DDT}(0, \alpha_1') > 0$ and $\texttt{GBCT}(\alpha_1, \alpha_1'; \beta, \beta) > 0$ is $(\alpha_1, \alpha_1') = (0, 0)$. Thus, $\texttt{iUBCT}(0, \beta) = 1$. The proof for $\texttt{iLBCT}(\alpha, 0) = 1$ is analogous.

For any α, choose an α_1 such that $\texttt{DDT}(\alpha, \alpha_1) > 0$, and set $\alpha_1' = \alpha_1$. The third condition, $\texttt{GBCT}(\alpha_1, \alpha_1; 0, 0) > 0$, simplifies to the identity $\alpha_1 = \alpha_1$, which is always

Table 7: iLBCT of SKINNY-64's S-box

$\alpha_0 \setminus \beta_2$	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	1	6	4	6	6	4	6	8	6	6	6	6	8	8	8	8
1	1	6	4	6	6	6	6	16	5	5	5	5	9	9	9	9
2	1	5	2	5	9	4	6	11	7	8	4	7	13	12	18	19
3	1	13	6	13	16	9	11	20	11	13	8	10	20	22	36	36
4	1	14	8	14	18	8	14	24	21	21	20	20	30	30	26	26
5	1	14	8	14	18	8	14	24	17	17	20	20	28	28	32	32
6	1	12	6	12	15	6	11	24	23	18	23	18	28	28	26	28
7	1	17	6	17	15	5	12	24	18	17	21	14	32	32	34	32
8	1	15	12	15	15	10	14	43	15	14	15	14	20	23	25	24
9	1	15	12	15	15	10	14	43	15	14	15	14	20	23	25	24
a	1	12	6	12	13	7	13	15	18	17	15	14	26	29	25	24
b	1	11	6	11	13	6	12	15	17	16	15	14	26	29	25	24
c	1	12	8	12	20	8	12	24	21	21	21	21	29	29	29	29
d	1	20	8	20	20	8	20	24	17	17	17	17	29	29	29	29
e	1	15	10	15	15	5	16	24	18	19	20	17	31	31	27	29
f	1	12	10	12	15	6	17	24	19	20	22	19	29	27	29	29

satisfied. Thus, at least one such pair (α_1, α_1) always exists, proving $\texttt{iUBCT}(\alpha, 0) > 0$. The proof for $\texttt{iLBCT}(0, \beta) > 0$ is analogous.

Proof of Property 2. The proof follows directly from the definitions and the known duality properties of the underlying tables for a function S and its inverse S^{-1}. According to Definition 6, $\texttt{iUBCT}_S(\alpha, \beta)$ counts the number of pairs (α_1, α_1') such that $\texttt{DDT}_S(\alpha, \alpha_1) > 0$, $\texttt{DDT}_S(\alpha, \alpha_1') > 0$, and $\texttt{GBCT}_S(\alpha_1, \alpha_1'; \beta, \beta) > 0$. Meanwhile, $\texttt{iLBCT}_{S^{-1}}(\beta, \alpha)$ counts pairs (β_1, β_1') where $\texttt{DDT}_{S^{-1}}(\beta_1, \alpha) > 0$, $\texttt{DDT}_{S^{-1}}(\beta_1', \alpha) > 0$, and $\texttt{GBCT}_{S^{-1}}(\beta, \beta; \beta_1, \beta_1') > 0$.

By applying the properties $\texttt{DDT}_S(a, b) = \texttt{DDT}_{S^{-1}}(b, a)$ and $\texttt{GBCT}_S(a, a'; b, b') = \texttt{GBCT}_{S^{-1}}(b, b'; a, a')$, it is clear that the set of conditions on (α_1, α_1') for $\texttt{iUBCT}_S$ is identical to the set of conditions on (β_1, β_1') for $\texttt{iLBCT}_{S^{-1}}$ if we let $\beta_1 = \alpha_1$ and $\beta_1' = \alpha_1'$. Since both functions count the number of elements in the same set of pairs, their values must be equal. The proof for the second statement, $\texttt{iLBCT}_S(\alpha, \beta) = \texttt{iUBCT}_{S^{-1}}(\beta, \alpha)$, is analogous.

D Related-Key Impossible Boomerang Key Recovery Attack in [26, 27]

In Figure 11, we provide an outline of the impossible boomerang attack. Suppose that $\Delta_{\text{I}} \nrightarrow \Delta_{\text{O}}$ over E_D is an impossible boomerang distinguisher and E_{B} and E_{F} are added by extending the distinguisher backward and forward. Suppose the input difference of the distinguisher Δ_{I} propagates backward with probability 1 to Δ_{B} over E_{B}^{-1}, the output difference of the distinguisher Δ_{O} propagates forward with probability 1 to Δ_{F} over E_{F}. Let V_{B} be the space spanned by all possible Δ_{B} where $r_{\text{B}} = \log_2 |V_{\text{B}}|$ denotes the dimension of the space. Let V_{F} be the space spanned by all possible Δ_{F} where $r_{\text{F}} = \log_2 |V_{\text{F}}|$ denotes the dimension of the space. Let c_{B} and c_{F} denote the number of

bit conditions satisfying $\Delta_{\mathrm{B}} \to \Delta_{\mathrm{I}}$ and $\Delta_{\mathrm{F}} \to \Delta_{\mathrm{O}}$, respectively. Let k_{B} and k_{F} be the subkey bits involved in E_{B} and E_{F} to verify $\Delta_{\mathrm{B}} \to \Delta_{\mathrm{I}}$ and $\Delta_{\mathrm{F}} \to \Delta_{\mathrm{O}}$, respectively.

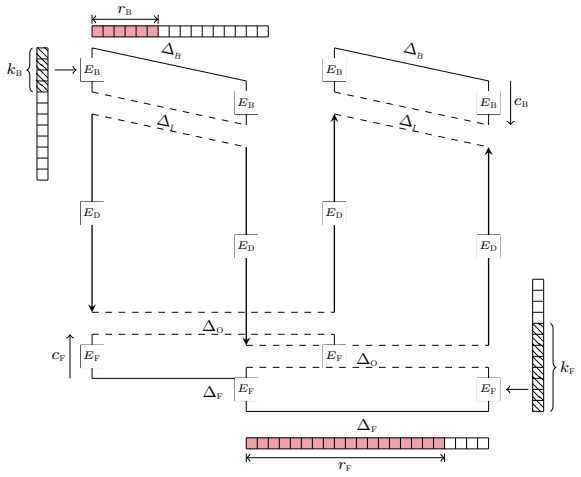


Fig. 11: Outline of the impossible boomerang attack

Here, we briefly recall the works in [26] and [27]; for detailed descriptions, please refer to the original papers. Method 1 refers to the Key Recovery with Quartet Filtering in [26] and the Impossible Differential Style in [27]. Method 2 refers to the Key Recovery with Pair Filtering in [26] and the Boomerang Style in [27]. Both methods are under related-key settings.

D.1 Method 1

Choose 2^y structures of $2^{r_{\mathrm{B}}}$ plaintexts each, we can construct $2^{y+2r_{\mathrm{B}}}$ pairs satisfying Δ_{B}. To go further, there would be $Q = 2^{2y+4r_{\mathrm{B}}+2r_{\mathrm{F}}-2n}$ quartets satisfying Δ_{B} and Δ_{F} constructed.

Thus the data complexity of the related-key IB attack is given by

$$D = 2^{y+r_{\mathrm{B}}+2}.$$

For the time complexity, it is composed of the time used for collecting quartets, guess-and-filter, and exhaustive search. In the phase of quartets collection, the time complexity is:

$$T_0 + T_1 = D + Q.$$

With the early abort technique [34], the time complexity for guess-and-filter phase would be bounded by

$$T_2 = 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|} \cdot \frac{Q}{2^{2c_{\mathrm{B}}+2c_{\mathrm{F}}}}.$$

The time complexity of the final phase, exhaustive search, is:

$$T_3 = 2^k \cdot P,$$

where P denotes the probability that a trial key is kept in the candidate keys:

$$P = (1 - 2^{-(2c_{\mathrm{B}}+2c_{\mathrm{F}})})^Q.$$

Let g be the number of key bits we can retrieve through the guess-and-filter phase, i.e., $P = 2^{-g}$, $1 < g \leq |k_{\mathrm{B}} \cup k_{\mathrm{F}}|$. Due to $(1 - 2^{-2(c_{\mathrm{B}}+c_{\mathrm{F}})})^Q \approx e^{-Q2^{-2(c_{\mathrm{B}}+c_{\mathrm{F}})}}$. Then we would have $Q = 2^{2c_{\mathrm{B}}+2c_{\mathrm{F}}+\log_2(g)-0.53}$. Let $\mathrm{LG}(g) = \log_2(g) - 0.53$, thus the time complexity of the IB attack can be reformulated as follows:

$$\begin{aligned} T_0 &= D, \\ T_1 &= 2^{2c_{\mathrm{B}}+2c_{\mathrm{F}}+\mathrm{LG}(g)}, \\ T_2 &= 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|+\mathrm{LG}(g)}, \\ T_3 &= 2^{k-g}. \end{aligned}$$

The total time complexity of the IB attack:

$$T = (T_0 + (T_1 + T_2)C_{E'} + T_3)C_E,$$

where C_E denotes the cost of one encryption and $C_{E'}$ is the ratio of the cost of partial encryption to the full encryption.

D.2 Method 2

Choose 2^y structures of $2^{r_{\mathrm{B}}}$ plaintexts each, thus the data complexity of the attack is given by

$$D = 2^{y+r_{\mathrm{B}}+2}.$$

Guess $2^{|k_{\mathrm{B}}|}$ possible values of k_{B}, we can obtain $2^{y+r_{\mathrm{B}}}$ pairs, which satisfy the difference Δ_{I} after encryption over E_{B}. To go further, there would be $Q = 2^{2y+2r_{\mathrm{B}}+2r_{\mathrm{F}}-2n}$ quartets under each guessed key satisfying Δ_{I} and Δ_{F} constructed. These Q quartets will be used in the guess-and-filter phase to eliminate incorrect keys.

For the time complexity, it is composed of the time used for collecting quartets, guess-and-filter, and exhaustive search. In the phase of quartets collection, the time complexity is:

$$T_0 = 2^{|k_{\mathrm{B}}|}2D$$

and

$$T_1 = 2^{|k_{\mathrm{B}}|}Q.$$

With the early abort technique [34], the time complexity for guess-and-filter phase would be bounded by

$$T_2 = 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|} \cdot \frac{Q}{2^{2c_{\mathrm{F}}}}.$$

The time complexity of the final phase, exhaustive search, is:

$$T_3 = 2^k \cdot P,$$

where P denotes the probability that a trial key is kept in the candidate keys:

$$P = (1 - 2^{-2c_{\mathrm{F}}})^Q.$$

Let g be the number of key bits we can retrieve through the guess-and-filter phase, i.e., $P = 2^{-g}$, $1 < g \leq |k_{\mathrm{B}} \cup k_{\mathrm{F}}|$. Due to $(1 - 2^{-2c_{\mathrm{F}}})^Q \approx e^{-Q2^{-2c_{\mathrm{F}}}}$. Then we would have $Q = 2^{2c_{\mathrm{F}}+\log_2(g)-0.53}$. Let $\mathrm{LG}(g) = \log_2(g) - 0.53$, thus the time complexity of the IB attack can be reformulated as follows:

$$\begin{aligned}
T_0 &= 2^{|k_{\mathrm{B}}|} \cdot 2D, \\
T_1 &= 2^{|k_{\mathrm{B}}|+2c_{\mathrm{F}}+\mathrm{LG}(g)}, \\
T_2 &= 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|+\mathrm{LG}(g)}, \\
T_3 &= 2^{k-g}.
\end{aligned}$$

And the memory complexity of the IB attack would be determined by $M = \min\{Q, 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|}\}$.

Then we would have the total time complexity of the IB attack:

$$T = (D + (T_0 + T_1 + T_2)C_{E'} + T_3)C_E,$$

where C_E denotes the cost of one encryption and $C_{E'}$ is the ratio of the cost of partial encryption to the full encryption.

E Complexity Analysis of the Related-(Twea)key ID Attack within the Generic Framework

In the related-(twea)key setting, the adversary has the ability to control the key differences and perform encryption/decryption of plaintext/ciphertext data using two keys, K_0 and $K_1 = K_0 \oplus \Delta K$. For the related-(twea)key impossible differential attack, due to the presence of key difference, plaintext data P in each plaintext structure can generate two distinct pairs $((P, K_0), (P \oplus \Delta_{\mathrm{B}}, K_1))$ and $((P \oplus \Delta_{\mathrm{B}}, K_0), (P, K_1))$ (distinguishing it from the single-(twea)key impossible differential attack). Under the related-(twea)key setting, the data complexity will include the ciphertext obtained from plaintext under two related keys. For the complexity analysis of related-(twea)key ID attacks, the data complexity and the time complexity of T_0 will change, while other terms remain.

The data complexity of the related-(twea)key ID attack is:

$$D = \max\left\{2^{\frac{c_{\mathrm{B}}+c_{\mathrm{F}}+n-r_{\mathrm{F}}+\mathrm{LG}(g)+x_1-c'_{\mathrm{B}}}{2}+1}, 2^{n+c_{\mathrm{B}}+c_{\mathrm{F}}-r_{\mathrm{B}}-r_{\mathrm{F}}+\mathrm{LG}(g)+1}\right\}.$$

The time complexity of the related-(twea)key ID attack is:

$$T_0 = 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}|} \cdot D,\ T_1 = 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| + c^*_{\mathrm{B}} + c^*_{\mathrm{F}} + \mathrm{LG}(g)},\ T_2 = 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}| + \mathrm{LG}(g)},\ T_3 = 2^{k-g}.$$

The memory complexity of the related-(twea)key ID attack is:

$$M = \min\{2^{c^*_{\mathrm{B}} + c^*_{\mathrm{F}} + \mathrm{LG}(g)}, 2^{|k^*_{\mathrm{B}} \cup k^*_{\mathrm{F}}|}\}$$

for storing the $N = 2^{c^*_{\mathrm{B}} + c^*_{\mathrm{F}} + \mathrm{LG}(g)}$ pairs or the discarded key candidates.

F Generic Key Recovery Framework for IB Attacks

In the single-key scenario, an IB attack can be transformed into an ID attack and is thus less interesting [25]. However, in the related-key setting, the flexibility in choosing key differences can give IB attacks an advantage over ID attacks [3, 25, 27]. We therefore present the generic key recovery framework for related-key IB attacks on SPN ciphers with a linear key schedule.

The key recovery process follows three main phases:

1. **Data Collection and Pre-filtering:** First, prepare 2^y structures of plaintexts, each of size $2^{r_{\mathrm{B}}}$. For each structure, query the ciphertexts under four related keys, yielding a total data complexity of $D = 4 \cdot 2^{y + r_{\mathrm{B}}}$. For each of the $2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}|}$ pre-guessed subkeys[4], perform partial encryption and decryption on all collected data to get intermediate states (P^*_i, C^*_i).

   The construction of quartets is a two-stage process. First, we form pairs satisfying the backward conditions, and then we match these pairs to form quartets satisfying the forward conditions. Following a "plaintext-first" approach:

   - *Step 1: Form Paired-Sets.* We create two distinct sets of pairs. The set S_1 is built from data corresponding to keys K_1 and K_2, containing pairs that satisfy the backward difference (c'_{B}-bit conditions). A similar set, S_2, is built using keys K_3 and K_4. The size of each set is approximately

   $$|S_1| = |S_2| = 2^y \cdot 2^{c'_{\mathrm{B}}} \cdot 2^{2r^*_{\mathrm{B}}} = (D/4) \cdot 2^{r^*_{\mathrm{B}}}$$

   - *Step 2: Match Pairs into Quartets.* We then use a hash table to match pairs from S_1 with pairs from S_2. A pair from S_1 is inserted into a hash table, indexed by the $2(n - r^*_{\mathrm{F}})$ unconstrained bits of its ciphertext components. A collision with a pair from S_2 signifies that the ciphertext differences align, forming a valid quartet.

   This procedure yields a total of Q candidate quartets, calculated as:

   $$Q \approx |S_1| \cdot |S_2| \cdot 2^{-2(n - r^*_{\mathrm{F}})} = ((D/4) \cdot 2^{r^*_{\mathrm{B}}})^2 \cdot 2^{-2(n - r^*_{\mathrm{F}})} = (D/4)^2 \cdot 2^{2r^*_{\mathrm{B}} + 2r^*_{\mathrm{F}} - 2n}$$

[4] Since the key schedule is linear, we do not differentiate between the guessed subkeys for each of the related keys.

Note. The procedure can also be performed with a "ciphertext-first" approach. The most efficient strategy is chosen to minimize the size of the intermediate sets that must be stored.

2. **Guess-and-Filter:** For each of the Q candidate quartets, guess the remaining subkey bits, $|k_{\mathrm{B}}^* \cup k_{\mathrm{F}}^*|$. Any subkey guess satisfying the impossible boomerang over the remaining $2c_{\mathrm{B}}^*$- and $2c_{\mathrm{F}}^*$-bit conditions is discarded. A wrong key survives with probability $P = (1 - 2^{-2(c_{\mathrm{B}}^* + c_{\mathrm{F}}^*)})^Q$. To achieve $P = 2^{-g}$, the required number of quartets Q is approximately:

$$Q \approx 2^{2c_{\mathrm{B}}^* + 2c_{\mathrm{F}}^* + \mathrm{LG}(g)}$$

3. **Exhaustive Search:** Finally, exhaustively search any remaining key candidates.

F.1 Complexity Analysis

The generic framework introduces a trade-off where an increased pre-computation cost leads to significant advantages in memory and potentially time complexity.

Memory Complexity. The primary advantage of this framework is the reduction in memory complexity. The memory required for the guess-and-filter phase is:

$$M = \min\{2^{2c_{\mathrm{B}}^* + 2c_{\mathrm{F}}^* + \mathrm{LG}(g)}, 2^{|k_{\mathrm{B}}^* \cup k_{\mathrm{F}}^*|}\}$$

By pre-guessing keys, we ensure that $c_{\mathrm{B}}^* + c_{\mathrm{F}}^* < c_{\mathrm{B}} + c_{\mathrm{F}}$ and $|k_{\mathrm{B}}^* \cup k_{\mathrm{F}}^*| < |k_{\mathrm{B}} \cup k_{\mathrm{F}}|$, which directly leads to a smaller memory complexity compared to classical methods. This reduction is demonstrated in our attacks on SKINNYee and Deoxys-BC in Sections 7, N.2 and N.3.

Time Complexity. The total time complexity is the sum of all phases, including the memory access (MA) cost for quartet generation:

$$T = (D + (T_{\text{pre-filter}} + T_{\text{filter}})C_{E'} + T_{\text{search}})C_E + T_{\text{quartet-gen}} \text{ (MAs)}$$

The components are:

- *Data Collection and Pre-filtering:* $D + T_{\text{pre-filter}} = D + 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}|} \cdot D$.
- *Quartet Generation:* $T_{\text{quartet-gen}} \approx 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}|} \cdot \min\{(D/4) \cdot 2^{r_{\mathrm{B}}^*}, (D/4)^2 \cdot 2^{r_{\mathrm{F}}^* - n}\}$[5].
- *Guess-and-Filter:*

$$\begin{aligned} T_{\text{filter}} = T_2 + T_3 &= 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}|} \cdot Q + 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}|} \cdot \frac{Q}{2^{2(c_{\mathrm{B}}^* + c_{\mathrm{F}}^*)}} \\ &= 2^{|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| + 2c_{\mathrm{B}}^* + 2c_{\mathrm{F}}^* + \mathrm{LG}(g)} + 2^{|k_{\mathrm{B}} \cup k_{\mathrm{F}}| + \mathrm{LG}(g)} \end{aligned}$$

[5]The first term in the minimum corresponds to a "plaintext-first" pair search, costing one memory access per pair. The second, "ciphertext-first" term is derived by finding collisions among the $D/4$ ciphertext states. These are placed into $2^{n - r_{\mathrm{F}}^*}$ hash bins, yielding approximately $(D/4)^2 \cdot 2^{r_{\mathrm{F}}^* - n}$ pairs satisfying the forward conditions.

- *Exhaustive Search:* $T_{\text{search}} = 2^{k-g}$.

This framework provides a notable potential for reducing the overall time complexity. A benefit is achieved if the condition $|k'_{\text{B}} \cup k'_{\text{F}}| < 2(c'_{\text{B}} + c'_{\text{F}})$ holds. Crucially, this condition is more likely to be satisfied in IB attacks than its counterpart in ID attacks. This is because a quartet involves two differential propagations, effectively doubling the number of verifiable bit-conditions ($2c'$) relative to the number of pre-guessed keys ($|k'|$). This provides significantly more filtering power, making a reduction in time complexity a tangible advantage of this framework for IB attacks.

Data Complexity. The data complexity is determined by the number of quartets, Q, needed for a successful attack, leading to:

$$D = 2^{n+c_{\text{B}}+c_{\text{F}}-r_{\text{B}}-r_{\text{F}}+\text{LG}(g)/2+2}$$

G Fast Identification of Zero Entries in iUBCT/iLBCT

In Algorithm 1, for a specific α_0, a pair (α_1, α'_1) that simultaneously satisfy $\texttt{DDT}(\alpha_0, \alpha'_1) > 0$ and $\texttt{DDT}(\alpha_0, \alpha_1) > 0$ would hold a probability of 2^{-2} approximately under the assumption of an S-box with differential uniformity 2. And for a specific β_2, a pair (α_1, α'_1) satisfying $\texttt{GBCT}(\alpha_1, \alpha'_1; \beta_2, \beta_2) > 0$ would hold a probability of 2^{-1} approximately with an experimental result for the target ciphers in this work. Then the computational complexity of the Algorithm 1 would be approximated by 2^{3n}. The computation of iLBCT in Algorithm 2 is similar to Algorithm 1. We have experimentally verified it for 4-bit and 8-bit Sboxes.

Algorithm 1: The algorithm for fast identification of zero entries in iUBCT

```
Construct the DDT with complexity of O(2^{2n}) and the GBCT(α, α'; β, β) with
  complexity of O(2^{3n}) (refer to the construction for BCT in [45]);
Initialize an empty table iUBCT with all 0 entries.
for all values of α ∈ F_2^n do
    for all values of β ∈ F_2^n do
        flag = False;
        for all values of α_1 ∈ F_2^n do
            for all values of α'_1 ∈ F_2^n do
                if DDT(α, α'_1) > 0 ∧ DDT(α, α_1) > 0 ∧ GBCT(α_1, α'_1; β, β) > 0
                 then
                    iUBCT(α, β) = 1;
                     flag=True;
                     break;
            if flag then
                break;
```

Algorithm 2: The algorithm for fast identification of zero entries in `iLBCT`

Construct the `DDT` with complexity of $\mathcal{O}(2^{2n})$ and the $\texttt{GBCT}(\alpha, \alpha; \beta, \beta')$ with complexity of $\mathcal{O}(2^{3n})$;
Initialize an empty table `iLBCT` with all 0 entries.
for *all values of* $\alpha \in \mathbb{F}_2^n$ **do**
  for *all values of* $\beta \in \mathbb{F}_2^n$ **do**
    flag = *False*;
    for *all values of* $\beta_1 \in \mathbb{F}_2^n$ **do**
      for *all values of* $\beta_1' \in \mathbb{F}_2^n$ **do**
        if $\texttt{DDT}(\beta_1, \beta) > 0 \wedge \texttt{DDT}(\beta_1', \beta) > 0 \wedge \texttt{GBCT}(\alpha, \alpha; \beta_1, \beta_1') > 0$ **then**
          $\texttt{iLBCT}(\alpha, \beta) = 1$;
          flag=*True*;
          break;
      if *flag* **then**
        break;

H `i3UBCT/i3MBCT/i3LBCT`: Extending `iUBCT/iLBCT` to 3 rounds

Definition 8 (`i3UBCT/i3MBCT/i3LBCT`)**.** *Let S be a bijective function over $\mathbb{F}_2^n$, and $\alpha, \beta \in \mathbb{F}_2^n$. The* `i3UBCT`*,* `i3MBCT` *and* `i3LBCT`*, with specific definitions provided as:*

$$\texttt{i3UBCT}(\alpha, \beta) = \# \left\{ (\alpha_1, \alpha_1', \alpha_2, \alpha_2') \in (\mathbb{F}_2^n)^4 \;\middle|\; \begin{array}{l} \texttt{DDT}(\alpha, \alpha_1) > 0, \texttt{DDT}(\alpha_1, \alpha_2) > 0, \\ \texttt{DDT}(\alpha, \alpha_1') > 0, \texttt{DDT}(\alpha_1', \alpha_2') > 0, \\ \texttt{GBCT}(\alpha_2, \alpha_2'; \beta, \beta) > 0 \end{array} \right\},$$

$$\texttt{i3MBCT}(\alpha, \beta) = \# \left\{ (\alpha_1, \alpha_1', \beta_2, \beta_2') \in (\mathbb{F}_2^n)^4 \;\middle|\; \begin{array}{l} \texttt{DDT}(\alpha, \alpha_1) > 0, \texttt{DDT}(\beta_2, \beta) > 0, \\ \texttt{DDT}(\alpha, \alpha_1') > 0, \texttt{DDT}(\beta_2', \beta) > 0, \\ \texttt{GBCT}(\alpha_1, \alpha_1'; \beta_2, \beta_2') > 0 \end{array} \right\},$$

$$\texttt{i3LBCT}(\alpha, \beta) = \# \left\{ (\beta_1, \beta_1', \beta_2, \beta_2') \in (\mathbb{F}_2^n)^4 \;\middle|\; \begin{array}{l} \texttt{DDT}(\beta_1, \beta_2) > 0, \texttt{DDT}(\beta_2, \beta) > 0, \\ \texttt{DDT}(\beta_1', \beta_2') > 0, \texttt{DDT}(\beta_2', \beta) > 0, \\ \texttt{GBCT}(\alpha, \alpha; \beta_1, \beta_1') > 0 \end{array} \right\},$$

I New 19-round Related-Tweakey Impossible Boomerang Distinguisher for SKINNY-128-384

We provide a 19-round related-tweakey impossible boomerang distinguisher for SKINNY-128-384 based on the `iLBCT` in Figure 12. All of the zero entries in the `iUBCT` and `iLBCT` of the SKINNY-128-384 S-box are listed in Table 8.

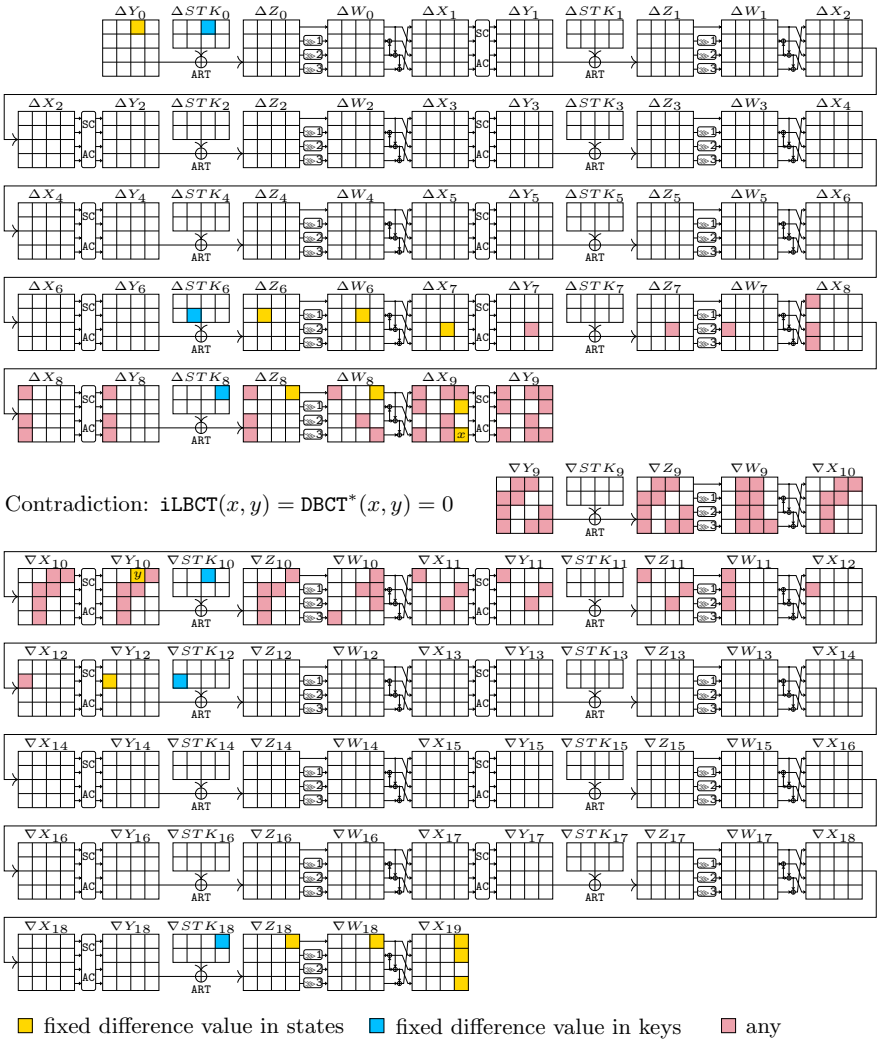


Fig. 12: 19-Round Related-Tweakey Impossible Boomerang Distinguisher for SKINNY-128-384

Table 8: The entries with zero value in the iUBCT and iLBCT of the SKINNY-128 S-box

iUBCT$(\alpha, \beta) = 0$
(0xa, 0x24), (0xa, 0x34), (0xa, 0xa4), (0xa, 0xb4), (0x20, 0x88), (0x20, 0x89), (0x20, 0x8c), (0x20, 0x8d), (0x20, 0xa8), (0x20, 0xa9), (0x20, 0xac), (0x20, 0xad), (0x40, 0x12), (0x40, 0x13), (0x40, 0x16), (0x40, 0x17), (0x40, 0x32), (0x40, 0x33), (0x40, 0x36), (0x40, 0x37), (0x50, 0x12), (0x50, 0x13), (0x50, 0x16), (0x50, 0x17), (0x50, 0x32), (0x50, 0x33), (0x50, 0x36), (0x50, 0x37), (0x60, 0x12), (0x60, 0x13), (0x60, 0x16), (0x60, 0x17), (0x60, 0x32), (0x60, 0x33), (0x60, 0x36), (0x60, 0x37), (0x70, 0x12), (0x70, 0x13), (0x70, 0x16), (0x70, 0x17), (0x70, 0x32), (0x70, 0x33), (0x70, 0x36), (0x70, 0x37), (0x80, 0xc2), (0x80, 0xc3), (0x80, 0xd2), (0x80, 0xd3), (0x90, 0xc2), (0x90, 0xc3), (0x90, 0xd2), (0x90, 0xd3), (0xc0, 0x12), (0xc0, 0x13), (0xc0, 0x16), (0xc0, 0x17), (0xc0, 0x32), (0xc0, 0x33), (0xc0, 0x36), (0xc0, 0x37), (0xd0, 0x12), (0xd0, 0x13), (0xd0, 0x16), (0xd0, 0x17), (0xd0, 0x32), (0xd0, 0x33), (0xd0, 0x36), (0xd0, 0x37), (0xe0, 0x12), (0xe0, 0x13), (0xe0, 0x16), (0xe0, 0x17), (0xe0, 0x32), (0xe0, 0x33), (0xe0, 0x36), (0xe0, 0x37), (0xf0, 0x12), (0xf0, 0x13), (0xf0, 0x16), (0xf0, 0x17), (0xf0, 0x32), (0xf0, 0x33), (0xf0, 0x36), (0xf0, 0x37)
iLBCT$(\alpha, \beta) = 0$
(0x2, 0x20), (0x2, 0x21), (0x2, 0x24), (0x2, 0x25), (0x4, 0x10), (0x4, 0x11), (0x4, 0x14), (0x4, 0x15), (0x5, 0x10), (0x5, 0x11), (0x5, 0x14), (0x5, 0x15), (0x12, 0x20), (0x12, 0x21), (0x12, 0x24), (0x12, 0x25), (0x14, 0x10), (0x14, 0x11), (0x14, 0x14), (0x14, 0x15), (0x15, 0x10), (0x15, 0x11), (0x15, 0x14), (0x15, 0x15), (0x17, 0x20), (0x20, 0x1), (0x20, 0x5), (0x20, 0x41), (0x20, 0x45), (0x20, 0x51), (0x20, 0x55), (0x20, 0xc1), (0x20, 0xc5), (0x20, 0xd1), (0x20, 0xd5), (0x22, 0x1), (0x22, 0x20), (0x22, 0x21), (0x22, 0x24), (0x22, 0x25), (0x22, 0x41), (0x22, 0x51), (0x24, 0x10), (0x24, 0x11), (0x24, 0x14), (0x24, 0x15), (0x25, 0x10), (0x25, 0x11), (0x25, 0x14), (0x25, 0x15), (0x32, 0x20), (0x32, 0x21), (0x32, 0x24), (0x32, 0x25), (0x34, 0x10), (0x34, 0x11), (0x34, 0x14), (0x34, 0x15), (0x35, 0x10), (0x35, 0x11), (0x35, 0x14), (0x35, 0x15), (0x37, 0x20), (0x42, 0x20), (0x42, 0x21), (0x42, 0x24), (0x42, 0x25), (0x44, 0x10), (0x44, 0x11), (0x44, 0x14), (0x44, 0x15), (0x45, 0x10), (0x45, 0x11), (0x45, 0x14), (0x45, 0x15), (0x4c, 0x10), (0x4c, 0x30), (0x4d, 0x10), (0x4d, 0x30), (0x4e, 0x10), (0x4f, 0x10), (0x52, 0x20), (0x52, 0x21), (0x52, 0x24), (0x52, 0x25), (0x54, 0x10), (0x54, 0x11), (0x54, 0x14), (0x54, 0x15), (0x55, 0x10), (0x55, 0x11), (0x55, 0x14), (0x55, 0x15), (0x5c, 0x10), (0x5c, 0x30), (0x5d, 0x10), (0x5d, 0x30), (0x5e, 0x10), (0x5f, 0x10), (0x62, 0x20), (0x62, 0x21), (0x62, 0x24), (0x62, 0x25), (0x64, 0x10), (0x64, 0x11), (0x64, 0x14), (0x64, 0x15), (0x65, 0x10), (0x65, 0x11), (0x65, 0x14), (0x65, 0x15), (0x6c, 0x10), (0x6c, 0x30), (0x6d, 0x10), (0x6d, 0x30), (0x6e, 0x10), (0x6f, 0x10), (0x72, 0x20), (0x72, 0x21), (0x72, 0x24), (0x72, 0x25), (0x74, 0x10), (0x74, 0x11), (0x74, 0x14), (0x74, 0x15), (0x75, 0x10), (0x75, 0x11), (0x75, 0x14), (0x75, 0x15), (0x7c, 0x10), (0x7c, 0x30), (0x7d, 0x10), (0x7d, 0x30), (0x7e, 0x10), (0x7f, 0x10), (0x82, 0x20), (0x82, 0x21), (0x82, 0x24), (0x82, 0x25), (0x84, 0x10), (0x84, 0x11), (0x84, 0x14), (0x84, 0x15), (0x85, 0x10), (0x85, 0x11), (0x85, 0x14), (0x85, 0x15), (0x92, 0x20), (0x92, 0x21), (0x92, 0x24), (0x92, 0x25), (0x94, 0x10), (0x94, 0x11), (0x94, 0x14), (0x94, 0x15), (0x95, 0x10), (0x95, 0x11), (0x95, 0x14), (0x95, 0x15), (0xa2, 0x20), (0xa2, 0x21), (0xa2, 0x24), (0xa2, 0x25), (0xa4, 0x10), (0xa4, 0x11), (0xa4, 0x14), (0xa4, 0x15), (0xa5, 0x10), (0xa5, 0x11), (0xa5, 0x14), (0xa5, 0x15), (0xb2, 0x20), (0xb2, 0x21), (0xb2, 0x24), (0xb2, 0x25), (0xb4, 0x10), (0xb4, 0x11), (0xb4, 0x14), (0xb4, 0x15), (0xb5, 0x10), (0xb5, 0x11), (0xb5, 0x14), (0xb5, 0x15), (0xc2, 0x20), (0xc2, 0x21), (0xc2, 0x24), (0xc2, 0x25), (0xc4, 0x10), (0xc4, 0x11), (0xc4, 0x14), (0xc4, 0x15), (0xc5, 0x10), (0xc5, 0x11), (0xc5, 0x14), (0xc5, 0x15), (0xd2, 0x20), (0xd2, 0x21), (0xd2, 0x24), (0xd2, 0x25), (0xd4, 0x10), (0xd4, 0x11), (0xd4, 0x14), (0xd4, 0x15), (0xd5, 0x10), (0xd5, 0x11), (0xd5, 0x14), (0xd5, 0x15), (0xe2, 0x20), (0xe2, 0x21), (0xe2, 0x24), (0xe2, 0x25), (0xe4, 0x10), (0xe4, 0x11), (0xe4, 0x14), (0xe4, 0x15), (0xe5, 0x10), (0xe5, 0x11), (0xe5, 0x14), (0xe5, 0x15), (0xf2, 0x20), (0xf2, 0x21), (0xf2, 0x24), (0xf2, 0x25), (0xf4, 0x10), (0xf4, 0x11), (0xf4, 0x14), (0xf4, 0x15), (0xf5, 0x10), (0xf5, 0x11), (0xf5, 0x14), (0xf5, 0x15)

J Supplementary Materials for the MILP Model Introduced in Section 6

J.1 Constraints for Deterministic Propagations in `SubBytes`

Let $y = S(x)$, then the constraints of deterministic propagation in the S-box can be written as:

$$\begin{cases} x^{s_0} - y^{s_0} = 0 \\ x^{s_1} - y^{s_1} \geq 0 \\ x^{s_0} - y^{s_1} \geq 0 \\ -x^{s_0} - x^{s_1} + y^{s_1} \geq -1 \end{cases}$$

J.2 Constraints for Deterministic Propagations in `MixColumns`

Let $z = x \oplus y$, then the constraints of deterministic propagation for the XOR operation can be written as:

$$\begin{cases} -x^{s_1} - y^{s_1} + z^{s_1} \geq -1 \\ x^{s_1} - z^{s_1} \geq 0 \\ y^{s_1} - z^{s_1} \geq 0 \\ -x^{s_0} - y^{s_0} + z^{s_0} \geq -1 \\ x^{s_1} + y^{s_0} + z^{s_0} \geq 1 \\ x^{s_0} + y^{s_1} + z^{s_0} \geq 1 \\ -x^{s_0} - x^{s_1} + y^{s_0} - z^{s_0} \geq -2 \\ x^{s_0} - y^{s_0} - y^{s_1} - z^{s_0} \geq -2 \end{cases}$$

J.3 Constraints for Probabilistic Propagations in `SubBytes`

Let $y = S(x)$, then the constraints of probabilistic propagation in the S-box can be written as:

$$\begin{cases} x^{s_0} - y^{s_0} = 0 \\ -x^{s_1} + y^{s_1} \geq 0 \\ x^{s_1} - y^{s_0} - y^{s_1} \geq -1 \end{cases}$$

J.4 Constraints for Probabilistic Propagations in `MixColumns`

Let $z = x \oplus y$, then the constraints of probabilistic propagation for the `XOR` operation can be written as:

$$\begin{cases} -2x^{s_0} - x^{s_1} - 2y^{s_0} - y^{s_1} + 2z^{s_0} + z^{s_1} \geq -3 \\ -x^{s_1} - y^{s_1} + z^{s_1} \geq -1 \\ x^{s_0} + y^{s_1} + z^{s_0} + z^{s_1} - 1 \geq 0 \\ x^{s_1} + y^{s_0} + z^{s_0} + z^{s_1} - 1 \geq 0 \\ -x^{s_0} - x^{s_1} + y^{s_0} - z^{s_0} \geq -2 \\ -x^{s_0} - x^{s_1} + y^{s_1} - z^{s_1} \geq -2 \\ x^{s_0} - y^{s_0} - y^{s_1} - z^{s_0} \geq -2 \\ x^{s_1} - y^{s_0} - y^{s_1} - z^{s_1} \geq -2 \end{cases}$$

J.5 Constraints for Identifying Pre-guessed Keys in the Data Collection Phase

The subkey cells need to be pre-guessed for verifing the conditions of Y_r can be calculated by

$$\texttt{if } Y_{r,i}^{up,v} = 1, \texttt{ then } K_{r,i}^{up,p} = 1,$$

and the subkey cells need to be pre-guessed for verifing the conditions of W_r can be calculated by

$$\texttt{if } \left((W_{r,i}^{up,v} = 1) \wedge (X_{r,j}^{up,s_0} + X_{r,j}^{up,s_1} \leq 1)\right) = 1, \texttt{ then } K_{r,j}^{up,p} = 1 \texttt{ for } \forall j \in {}^{t}L^{D}(i).$$

In addition, we can backtrack to calculate all the pre-guessed subkey cells from round 0 using

$$\texttt{if } K_{r,i}^{up,p} = 1, \texttt{ then } K_{r-1,j}^{up,p} = 1 \texttt{ for } \forall j \in {}^{t}L^{D}(i).$$

J.6 Linear Layer Matrix L_E and L_D used in Section 6.1

For SKINNY-family,

$$L^E = \begin{pmatrix}
1&0&0&0&1&0&0&0&0&0&0&0&1&0&0&0\\
0&1&0&0&0&1&0&0&0&0&0&0&0&1&0&0\\
0&0&1&0&0&0&1&0&0&0&0&0&0&0&1&0\\
0&0&0&1&0&0&0&1&0&0&0&0&0&0&0&1\\
0&0&0&0&0&0&0&0&1&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&1&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&1&0&0&0&0&0\\
0&0&0&0&0&0&0&0&1&0&0&0&0&0&0&0\\
0&0&1&0&0&0&0&0&0&1&0&0&0&1&0&0\\
0&0&0&1&0&0&0&0&0&0&1&0&0&0&1&0\\
1&0&0&0&0&0&0&0&1&0&0&0&1&0&0&0\\
0&1&0&0&0&0&0&0&0&1&0&0&0&1&0&0\\
0&0&0&1&0&0&0&0&0&0&0&0&0&0&0&0\\
1&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&1&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&1&0&0&0&0&0&0&0&0&0&0&0&0&0
\end{pmatrix}$$

$$L^D = \begin{pmatrix}
0&0&0&0&0&0&0&0&0&0&0&0&0&1&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&1&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&1\\
0&0&0&0&0&0&0&0&0&0&0&0&1&0&0&0\\
1&0&0&0&0&0&0&1&0&0&1&0&0&0&0&0\\
0&1&0&0&1&0&0&0&0&0&0&1&0&0&0&0\\
0&0&1&0&0&1&0&0&1&0&0&0&0&0&0&0\\
0&0&0&1&0&0&1&0&0&1&0&0&0&0&0&0\\
0&0&0&0&0&0&0&1&0&0&0&0&0&0&0&0\\
0&0&0&0&1&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&1&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&1&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&1&0&0&1&0&0&1&0&0\\
0&0&0&0&1&0&0&0&0&0&0&1&0&0&1&0\\
0&0&0&0&0&1&0&0&1&0&0&0&0&0&0&1\\
0&0&0&0&0&0&1&0&0&1&0&0&1&0&0&0
\end{pmatrix}$$

For AES and Deoxys-BC,

$$L^E = \begin{pmatrix}
1&1&1&1&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&1&1&1&1\\
0&0&0&0&0&0&0&0&1&1&1&1&0&0&0&0\\
0&0&0&0&1&1&1&1&0&0&0&0&0&0&0&0\\
0&0&0&0&1&1&1&1&0&0&0&0&0&0&0&0\\
1&1&1&1&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&1&1&1&1\\
0&0&0&0&0&0&0&0&1&1&1&1&0&0&0&0\\
0&0&0&0&0&0&0&0&1&1&1&1&0&0&0&0\\
0&0&0&0&1&1&1&1&0&0&0&0&0&0&0&0\\
1&1&1&1&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&1&1&1&1\\
0&0&0&0&0&0&0&0&0&0&0&0&1&1&1&1\\
0&0&0&0&0&0&0&0&1&1&1&1&0&0&0&0\\
0&0&0&0&1&1&1&1&0&0&0&0&0&0&0&0\\
1&1&1&1&0&0&0&0&0&0&0&0&0&0&0&0
\end{pmatrix}$$

$$L^D = \begin{pmatrix}
1&0&0&0&0&1&0&0&0&0&1&0&0&0&0&1\\
1&0&0&0&0&1&0&0&0&0&1&0&0&0&0&1\\
1&0&0&0&0&1&0&0&0&0&1&0&0&0&0&1\\
1&0&0&0&0&1&0&0&0&0&1&0&0&0&0&1\\
0&0&0&1&1&0&0&0&0&1&0&0&0&0&1&0\\
0&0&0&1&1&0&0&0&0&1&0&0&0&0&1&0\\
0&0&0&1&1&0&0&0&0&1&0&0&0&0&1&0\\
0&0&0&1&1&0&0&0&0&1&0&0&0&0&1&0\\
0&0&1&0&0&0&0&1&1&0&0&0&0&1&0&0\\
0&0&1&0&0&0&0&1&1&0&0&0&0&1&0&0\\
0&0&1&0&0&0&0&1&1&0&0&0&0&1&0&0\\
0&0&1&0&0&0&0&1&1&0&0&0&0&1&0&0\\
0&1&0&0&0&0&1&0&0&0&0&1&1&0&0&0\\
0&1&0&0&0&0&1&0&0&0&0&1&1&0&0&0\\
0&1&0&0&0&0&1&0&0&0&0&1&1&0&0&0\\
0&1&0&0&0&0&1&0&0&0&0&1&1&0&0&0
\end{pmatrix}$$

J.7 Algorithm for the optimization and verification procedure of the model

The algorithm for the optimization and verification procedure of the model is provided in algorithm 3.

K Specification of SKINNYee

SKINNYee is a tweakable block cipher derived from the SKINNY family [40], proposed by Naito *et al.* at CRYPTO 2022 [46]. SKINNYee conforms to the new security scheme HOMA proposed in [46], featuring a 64-bit block size, a 128-bit key size, a 259-bit tweak size, and a total of 56 rounds. The design of SKINNYee is based on SKINNYe (TK4 setting) [47]. The round function is quite similar to that of the SKINNY family, but there are some differences in the operation `AddRoundKey` and the round constant generation. Figure 13 provides the round transformation of SKINNYee, and for more detailed specification please refer to the original design document.

The $LFSR_2$, $LFSR_3$, $LFSR_4$ and the cell permutation P_T are defined as:

$$\begin{aligned}
&LFSR_2 : (x_3 \|x_2\| x_1\|x_0) \rightarrow (x_2 \|x_1\| x_0\|x_3 \oplus x_2)\\
&LFSR_3 : (x_3 \|x_2\| x_1\|x_0) \rightarrow (x_0 \oplus x_3 \|x_3\| x_2\|x_1)\\
&LFSR_4 : (x_3 \|x_2\| x_1\|x_0) \rightarrow (x_1 \|x_0\| x_3 \oplus x_2\|x_2 \oplus x_1)\\
&P_T : (0, ..., 15) \rightarrow (9, 15, 8, 13, 10, 14, 12, 11, 0, 1, 2, 3, 4, 5, 6, 7)
\end{aligned}$$

Algorithm 3: Optimizing Model and Verifying Contradictions

Input: model $\mathcal{M}$, number of target rounds r, state variables $X_0...X_{r-1}, Y_0...Y_{r-1}, Z_0...Z_{r-1}, W_0...W_{r-1}$, key variables $K_0...K_{r-1}$, contradiction variables $\texttt{contr}_{1,2,3}$
Output: minimum time complexity T
Initialize: $flag = 0$
while $flag = 0$ **do**
  Add constraints on the round functions to model $\mathcal{M}$
  Add constraints on the involved subkey cells to model $\mathcal{M}$
  Add constraints on the contradictions to model $\mathcal{M}$
  Compute r'_{B}, r'_{F}, c'_{B}, c'_{F}, k'_{B}, k'_{F} and add to model $\mathcal{M}$
  $\mathcal{M} \leftarrow \bigvee_{i=1}^{3} \texttt{contr}_{\texttt{i}} = 1$
  $T \leftarrow \mathcal{M}.sol()$
  for each i **do**
    if $\texttt{contr}_{\texttt{i}} = 1$ **then**
      Verify the contradiction with $\texttt{BCT}(\texttt{contr}_1 = 1)$, $\texttt{iUBCT/iLBCT}(\texttt{contr}_2 = 1)$, and $\texttt{i3UBCT/i3MBCT/i3LBCT}(\texttt{contr}_3 = 1)$
      if an instance exists **then**
        $flag = 1$
        break
      else
        $\mathcal{M} \leftarrow \texttt{contr}_{\texttt{i}} = 0$
      end if
    end if
  end for
end while
return T

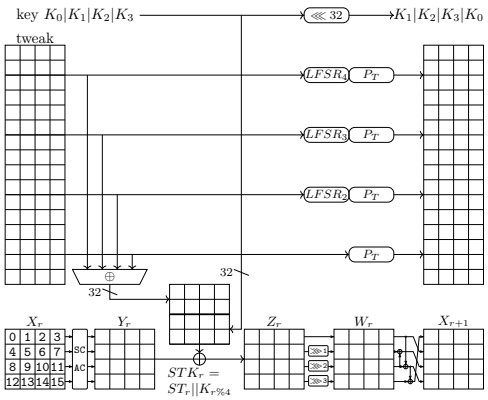


Fig. 13: Round Transformation of SKINNYee

L Applications to SKINNY-family

L.1 Specification

SKINNY is a tweakable block cipher family following the TWEAKEY framework, first proposed by Beierle *et al.* at CRYPTO 2016 [40]. SKINNY family has 6 versions, denoted by SKINNY-n-t: $n \in \{64, 128\}$ is the block size and $t \in \{n, 2n, 3n\}$ is the tweakey size. The cell size c is 4 for $n = 64$ and 8 for $n = 128$.

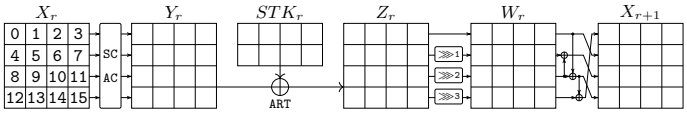


Fig. 14: Round function of SKINNY

The SKINNY round function (see Figure 14) applies five transformations: `SubCells` (`SC`), `AddConstants` (`AC`), `AddRoundTweakey` (`ART`), `ShiftRows` (`SR`), `MixColumns` (`MC`). The `SC` operation applies a 4-bit (resp. 8-bit) S-box on each cell for SKINNY-64 (resp. SKINNY-128). The `AC` operation XORs the round constant to the internal state. The `ART` operation XORs the first and second rows of subtweakey with the corresponding cells in the internal state. The `SR` operation rotates the 4-cell i-th row right by i positions, $i = 0, 1, 2, 3$. The `MC` operation multiplies the internal state by a binary matrix

$$\mathrm{M} = \begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}.$$

The tweakey schedule of SKINNY is a linear algorithm, which divides the master tweakey into z tweakey arrays $(TK1, ..., TKz)$ with n-bit length each, where $z = \frac{t}{n} \in \{1, 2, 3\}$. $TK1$, $TK2$ and $TK3$ follow three independent update functions. The subtweakey used in r-th round STK_r is generated from:

- $STK_r = TK1_r$ if $z = 1$,
- $STK_r = TK1_r \oplus TK2_r$ if $z = 2$,
- $STK_r = TK1_r \oplus TK2_r \oplus TK3_r$ if $z = 3$,

where $TK1_r, TK2_r, TK3_r$ denote the tweakey arrays in round r and are generated as follows. First, a permutation h is applied to each tweakey array as $TKz_{r+1}[i] \leftarrow TKz_r[h[i]]$. Next, each cell of the first and second rows of $TK2_r$ and $TK3_r$ are individually updated with an LFSR. For more details on the specification of SKINNY, please refer to [40].

L.2 Related-Tweakey Impossible Differential Attack on SKINNY-n-n

This section provides a 19-round related-tweakey impossible differential attack against SKINNY-n-n. The specification of SKINNY is provided in Appendix L.1. In this attack, we use a 12-round related-tweakey impossible differential as:

$$\begin{aligned}(\Delta Y_3, \Delta STK_3) &= ((00a0|0000|a000|00?0), (0000|0a00|0000|0000)) \\ &\nrightarrow (\Delta X_{16}) = (0000|0000|0a00|0000),\end{aligned}$$

where a denotes a fixed non-zero difference and ? denotes any difference. We prefix 4 rounds at the beginning and append 3 rounds at the end of the distinguisher to mount the attack, as shown in Figure 15. From the figure, we can get the parameters used for this attack: $r_{\text{B}} = 8c$, $c_{\text{B}} = 7c$, $r_{\text{F}} = 6c$, $c_{\text{F}} = 6c$, $|k_{\text{B}} \cup k_{\text{F}}| = 13c$, $c'_{\text{B}} = 0$, $c'_{\text{F}} = 0$, $|k'_{\text{B}} \cup k'_{\text{F}}| = 0$. In the key-recovery extensions of this attack, it adopts deterministic extensions.

We use Lemma 2 in the complexity analysis of our attacks.

Lemma 2. *For a given S-box and any nonzero input-output difference pair (δ_i, δ_o), there would exist one solution x on average for the equation $S(x) \oplus S(x \oplus \delta_i) = \delta_o$ holds true.*

Pairs Collection. In this phase, we need to collect $N = 2^{c_{\text{B}}+c_{\text{F}}+\text{LG}(g)} = 2^{13c+\text{LG}(g)}$ pairs to eliminate the wrong keys. **Guess-and-Filter.** For N pairs:

1. *Satisfying the cell conditions in ΔW_{17}.* From the ciphertexts, we can know the value of $X_{18}[8-15]$. With the condition $\Delta W_{17}[13] = \Delta X_{18}[1] \oplus \Delta X_{18}[13] = 0$, we can deduce the difference value $\Delta X_{18}[1]$. With known $\Delta Y_{18}[1]$ and Lemma 2, we can compute $X_{18}[1]$, $Y_{18}[1]$ and $STK_{18}[1]$. Similarly, we can also compute $X_{18}[3,7]$, $Y_{18}[3,7]$ and $STK_{18}[3,7]$. The time complexity of this step is $N \cdot 2 \cdot \frac{3}{19 \cdot 16} = N2^{-5.66}$.
2. *Satisfying the cell conditions in ΔW_{16}.* Guess 2^c possible values of $STK_{18}[2]$ and compute $X_{17}[15]$ and $Y_{17}[15]$. With the condition $\Delta W_{16}[15] = \Delta X_{17}[3] \oplus \Delta X_{17}[15] = 0$, we can use Lemma 2 and known $\Delta Y_{17}[3]$ to deduce $STK_{17}[3]$. Similarly, from the condition $\Delta W_{16}[7] = \Delta X_{17}[11] \oplus \Delta X_{17}[15] = 0$, we can deduce $STK_{18}[5]$. The time complexity of this step is $2^c \cdot N \cdot 2 \cdot \frac{3}{19 \cdot 16} = N2^{c-5.66}$.
3. *Satisfying the cell conditions in ΔX_2.* Due to $eSTK_0[4] = STK_{18}[2]$, $eSTK_0[11] = STK_{18}[5]$, we can compute $\Delta W_1[5,9]$. Checking whether $\Delta W_1[5] = \Delta W_1[9]$ or not would be a c-bit filter. With the condition $\Delta X_2[13] = \Delta W_1[1] \oplus \Delta W_1[9] = 0$, we can compute the value of $\Delta W_1[1]$ and then compute $X_1[1]$, $Y_1[1]$ and $eSTK_0[1]$ by Lemma 2. Time of this step is $2^c \cdot N \cdot 2^{-c} \cdot 2 \cdot \frac{1}{19 \cdot 16} \approx N2^{-7.25}$.
4. *Satisfying the cell conditions in ΔX_2.* Due to $eSTK_0[3] = STK_{18}[7]$, $eSTK_0[12] = STK_{18}[2]$, we can compute $\Delta W_1[3,15]$. With the condition $\Delta X_2[3] = \Delta W_1[3] \oplus \Delta W_1[11] \oplus \Delta W_1[15] = 0$, we can compute $\Delta W_1[11]$ and deduce $eSTK_0[9]$ by Lemma 2. Similarly, we can also deduce $eSTK_0[6]$ from the condition $\Delta X_2[11] = \Delta W_1[7] \oplus \Delta W_1[11] = 0$. The time complexity of this step is $2^c \cdot N \cdot 2^{-c} \cdot 2 \cdot \frac{4}{19 \cdot 16} = N2^{-5.25}$.

Fig. 15: The related-tweakey impossible differential attack against 19-round SKINNY-n-n

5. *Satisfying the cell conditions in* ΔX_{16}. From the previous steps, we can get all the values of $STK_{18}[1-5,7], STK_{17}[3]$ and thus $Y_{17}[15], \Delta Y_{17}[7], Z_{17}[7,15]$. Due to $\Delta Y_{16}[9] = \Delta X_{17}[15]$, we can deduce $Y_{16}[9]$ by Lemma 2, and then compute $STK_{17}[7]$. The time complexity of this step is $2^c \cdot N \cdot 2^{-c} \cdot 2 \cdot \frac{1}{19 \cdot 16} = N2^{-7.25}$.
6. *Satisfying the cell conditions in* ΔX_3. Guess $eSTK_0[1]$. With $STK_1[3] = STK_{17}[3]$ and $STK_1[7] = STK_{17}[7]$, we can compute $X_2[8]$ and $X_2[3,7,15]$. From the condition $\Delta X_3[10] = \Delta W_2[6] \oplus \Delta W_2[10] = 0$, we would have one solution $X_2[5]$ from the equation of S-box by Lemma 2 and deduce $STK_1[1]$. The time complexity of this step is $2^{2c} \cdot N \cdot 2^{-c} \cdot 2 \cdot \frac{2}{19 \cdot 16} = N2^{c-6.25}$.

7. *Satisfying the cell conditions in* ΔY_3. From the previous steps, we can get the values of $W_2[4]$ and $\Delta X_3[8]$. With $\Delta Y_3[8] = a$, one solution $X_3[8]$ could be derived from Lemma 2. Then we can compute $W_2[8]$ and $X_2[10]$. Due to $X_2[10] = Z_1[5] \oplus Z_1[8]$, we would have the actual value of $Z_1[5]$ and then compute $STK_1[5]$. Similarly, $STK_1[2]$ could also be computed. The time complexity of this step is $2^{2c} \cdot N \cdot 2^{-c} \cdot 2 \cdot \frac{2}{19 \cdot 16} = N2^{c-6.25}$.

Complexity. The data complexity is $D = 2^{15c+\mathrm{LG}(g)+1}$. The time complexity is $2^{15c+\mathrm{LG}(g)+1} + 2^{14c-4.44+\mathrm{LG}(g)} + 2^{16c-g}$. Memory complexity is 2^{13c}. For SKINNY-64-64 with $c = 4$, we set $g = 2$, then $D = 2^{61.47}$, $T = 2^{62.76}$, $M = 2^{52}$. For SKINNY-128-128 with $c = 8$, we set $g = 4$, then $D = 2^{122.47}$, $T = 2^{124.43}$, $M = 2^{104}$.

L.3 Single-Tweakey Impossible Differential Attack on SKINNY-n-2n

In this section, we propose a 19-round single-tweakey impossible differential attack against SKINNY-n-2n. In this attack, we use a 12-round single-tweakey impossible differential as:

$$\Delta X_3 = (0000|0000|0000|000?) \nrightarrow \Delta W_{14} = (0000|0000|0?00|0000).$$

We prefix 3 rounds at the beginning and append 4 rounds at the end of the distinguisher to mount the attack, as shown in Figure 16. From the figure, we can get the parameters used for this attack: $r_{\mathrm{B}} = 7c$, $c_{\mathrm{B}} = 6c$, $r_{\mathrm{F}} = 13c$, $c_{\mathrm{F}} = 13c$, $|k_{\mathrm{B}} \cup k_{\mathrm{F}}| = 25c$, $c'_{\mathrm{B}} = 0$, $c'_{\mathrm{F}} = 3c$, $|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| = 8c$. In the key-recovery extensions of this attack, it adopts probabilistic extensions.

Pairs Collection. In this phase, we need to collect $N = 2^{c^*_{\mathrm{B}} + c^*_{\mathrm{F}} + \mathrm{LG}(g)} = 2^{16c+\mathrm{LG}(g)}$ pairs under 2^{8c} pre-guessed subkey bits to eliminate the wrong keys. **Guess-and-Filter.** For N pairs under each pre-guessed subkey bits:

1. *Satisfying the cell conditions in* ΔW_{16}. With condition $\Delta W_{16}[4] = \Delta X_{17}[4] \oplus \Delta X_{17}[12] = 0$, we can compute $STK_{17}[4]$ and $X_{17}[4]$ by Lemma 2. Similarly, we could deduce the value of $STK_{17}[3, 5, 7]$. Time complexity of this step is $2^{8c} \cdot N \cdot 2 \cdot \frac{4}{19 \cdot 16} = N2^{8c-5.25}$.
2. *Satisfying the cell conditions in* ΔW_{15}. Guess $STK_{17}[2]$, we will have a c-bit filter for the condition $\Delta W_{15}[7] = \Delta X_{16}[11] \oplus \Delta X_{16}[15] = 0$. Then with the condition $\Delta W_{15}[15] = \Delta X_{16}[3] \oplus \Delta X_{16}[15] = 0$ and Lemma 2, we would derive the value of $STK_{16}[3]$. Similarly, we would compute the value of $STK_{16}[1, 5]$ by guessing $STK_{17}[0, 6]$ and using Lemma 2. Time complexity of this step is $2^{9c} \cdot N \cdot 2 \cdot \frac{1}{19 \cdot 16} + 2^{11c} \cdot N \cdot 2^{-c} \cdot 2 \cdot \frac{4}{19 \cdot 16} \approx N2^{10c-5.25}$.
3. *Satisfying the cell conditions in* ΔX_2. From previous steps, we have known the value of $STK_{16}[1, 3, 5]$ and $STK_{18}[0, 3, 7]$. With the tweakey schedule of SKINNY-n-2n, we can compute $STK_0[1, 3, 5] = eSTK_0[13, 10, 7]$. Then we can compute $Y_1[7, 10, 13]$

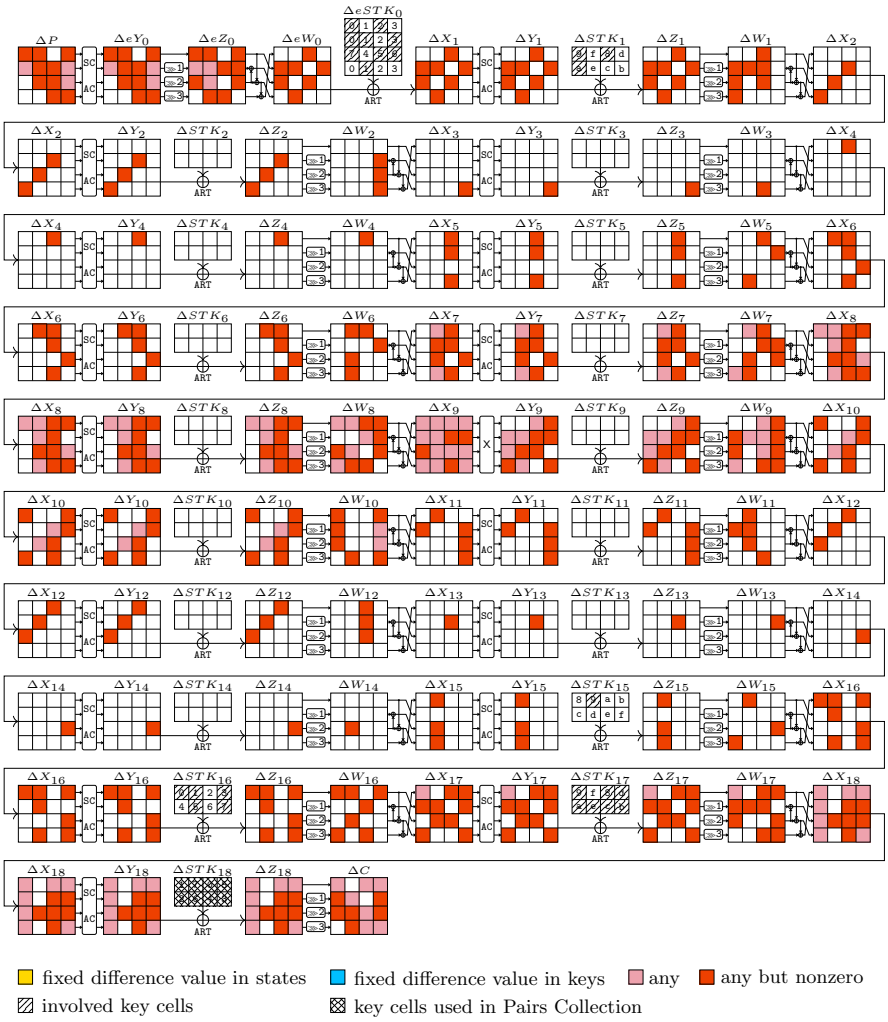


Fig. 16: The single-tweakey impossible differential attack against 19-round SKINNY-n-2n

and the condition $\Delta W_1[4] = \Delta W_1[8] = \Delta W_1[12]$ will lead to 2 c-bit filters. Time cost of this step would be a negligible one compared with previous steps.

4. *Satisfying the cell conditions in* ΔW_{14}. Guess $STK_{16}[0,7]$, and we will know the values of cells in Z_{15}. The condition $\Delta W_{14}[5] = \Delta X_{15}[9] \oplus \Delta X_{15}[13] = 0$ will lead to a c-bit filter. Meanwhile, we would determine $STK_{15}[1]$ by applying Lemma 2. Time complexity of this step is $2^{13c} \cdot N \cdot 2^{-3c} \cdot 2 \cdot \frac{2}{19 \cdot 16} = N2^{10c-6.25}$.
5. *Satisfying the cell conditions in* ΔX_2. We can compute $eSTK_0[5,8]$ from the tweakey schedule and known subtweakeys in previous steps, thus the condition $\Delta X_2[10] = \Delta Z_1[5] \oplus \Delta Z_1[8]$ will lead to a c-bit filter. Meanwhile, we would

determine $eSTK_0[2]$ by applying Lemma 2. Time complexity of this step is $2^{13c} \cdot N \cdot 2^{-4c} \cdot 2 \cdot \frac{1}{19 \cdot 16} = N2^{9c-7.25}$.

6. *Satisfying the cell conditions in* ΔX_3. Guess $eSTK_0[11]$ and $STK_1[4]$. From previous steps, we have known $STK_{17}[0]$ and $STK_{15}[1]$, thus we can compute $STK_1[1]$ by tweakey schedule and $Y_2[12]$. The condition $\Delta X_3[3] = \Delta Y_2[9] \oplus \Delta Y_2[12] = 0$ will lead to a c-bit filter. With condition $\Delta Y_2[6] = \Delta Y_2[12]$ and $\Delta X_2[6] = \Delta W_1[2]$, we can deduce $X_2[6]$ and $STK_1[2]$ by applying Lemma 2. Time complexity of this step is $2^{15c} \cdot N \cdot 2^{-5c} \cdot 2 \cdot \frac{2}{19 \cdot 16} = N2^{10c-6.25}$.

Complexity. The data complexity is $D = 2^{15c+\mathrm{LG}(g)+1}$. The time complexity of this whole attack is $2^{8c+15c+\mathrm{LG}(g)+1} \cdot \frac{8}{19 \cdot 16} + 2^{16c+10c+\mathrm{LG}(g)-4.25} + 2^{32c-g}$. For SKINNY-64-128 with $c = 4$, we set $g = 24$, then $D = 2^{65.05}$, $T = 2^{104.90}$, $M = 2^{68.05}$. For SKINNY-128-256 with $c = 8$, we set $g = 48$, then $D = 2^{126.05}$, $T = 2^{209.45}$, $M = 2^{133.05}$.

L.4 Single-Tweakey Impossible Differential Attack on SKINNY-n-3n

In this section, we propose a 21-round single-tweakey impossible differential attack against SKINNY-n-3n. In this attack, we use a 12-round single-tweakey impossible differential as:

$$\Delta X_5 = (0000|0000|0000|000?) \nrightarrow \Delta W_{16} = (0000|0000|0?00|0000).$$

We prefix 5 rounds at the beginning and append 4 rounds at the end of the distinguisher to mount the attack, as shown in Figure 17. From the figure, we can get the parameters used for this attack: $r_{\mathrm{B}} = 16c$, $c_{\mathrm{B}} = 15c$, $r_{\mathrm{F}} = 13c$, $c_{\mathrm{F}} = 13c$, $|k_{\mathrm{B}} \cup k_{\mathrm{F}}| = 41c$, $c'_{\mathrm{B}} = 0$, $c'_{\mathrm{F}} = 3c$, $|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| = 8c$. In the key-recovery extensions of this attack, it adopts probabilistic extensions.

Pairs Collection. In this phase, we need to collect $N = 2^{c^*_{\mathrm{B}}+c^*_{\mathrm{F}}+\mathrm{LG}(g)} = 2^{25c+\mathrm{LG}(g)}$ pairs under 2^{8c} pre-guessed subkey bits.

Guess-and-Filter. For N pairs under each pre-guessed subkey bits:

1. *Satisfying the cell conditions in* ΔW_{18}. With condition $\Delta W_{18}[4] = \Delta X_{19}[4] \oplus \Delta X_{19}[12]$, we can compute $STK_{19}[4]$ and $X_{19}[4]$ by Lemma 2. Similarly, we could deduce the value of $STK_{19}[3, 5, 7]$. Time complexity of this step is $2^{8c} \cdot N \cdot 2 \cdot \frac{4}{21 \cdot 16} = N2^{8c-5.39}$.
2. *Satisfying the cell conditions in* ΔX_2. Guess $eSTK_0[0{-}3, 8{-}11]$ and compute ΔW_1. The conditions $\Delta W_1[0] = \Delta W_1[4] = \Delta W_1[8]$, $\Delta W_1[1] = \Delta W_1[9]$ and $\Delta W_1[2] \oplus \Delta W_1[10] = \Delta W_1[14]$ will lead to four c-bit filters. Time complexity of this step is $2^{16c} \cdot N \cdot 2 \cdot \frac{8}{21 \cdot 16} = N2^{16c-4.39}$.
3. *Satisfying the cell conditions in* ΔX_3. Guess $STK_1[0, 1, 2, 6]$ and compute $Y_2[1, 4, 11, 14]$, and so $\Delta Y_2[1, 4, 11, 14]$. The conditions $\Delta Y_2[4] = \Delta Y_2[11]$ and $\Delta Y_2[1] = \Delta Y_2[11] \oplus \Delta Y_2[14]$ will lead to two c-bit filters. Guess $STK_1[3, 4, 5]$ and

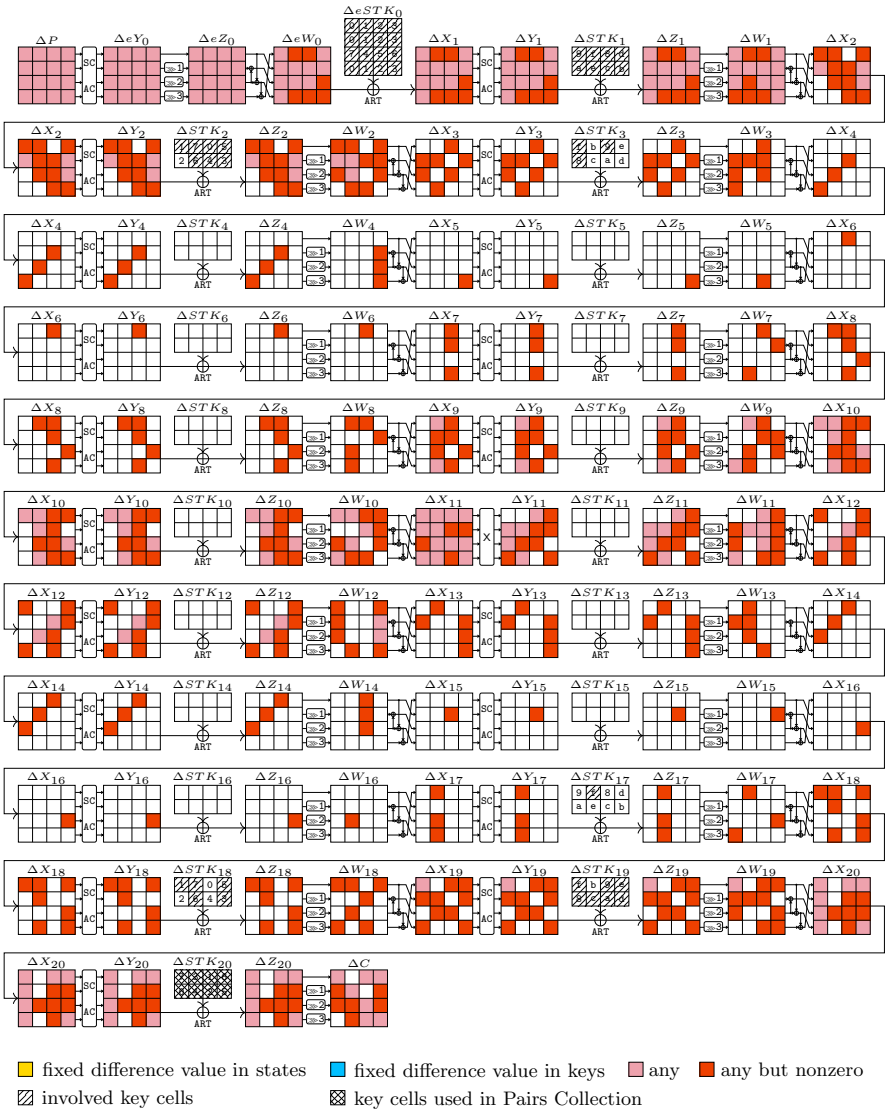


Fig. 17: The single-tweakey impossible differential attack against 21-round SKINNY-n-3n

compute $Y_2[0,3,6,9,10]$. The conditions $\Delta Y_2[0] = \Delta Y_2[10]$ and $\Delta Y_2[3] = \Delta Y_2[6] = \Delta Y_2[9]$ will lead to three c-bit filters. Guess $STK_1[7]$. Time complexity of this step could be approximated by $2^{20c} \cdot N \cdot 2^{-4c} \cdot 2 \cdot \frac{4}{21 \cdot 16} + 2^{23c} \cdot N \cdot 2^{-6c} \cdot 2 \cdot \frac{3}{21 \cdot 16} + 2^{24c} \cdot N \cdot 2^{-9c} \cdot 2 \cdot \frac{1}{21 \cdot 16} \approx N 2^{17c-5.81}$.

4. *Satisfying the cell conditions in* ΔW_{17}. Guess $STK_{19}[2]$ and compute $\Delta X_8[11,15]$, which will lead to a c-bit filter. Guess $STK_{19}[0,6]$. With condition $\Delta W_{17}[15] =$

$\Delta X_{18}[3] \oplus \Delta X_{18}[15] = 0$, we can determine $STK_{18}[3]$ by applying Lemma 2. Similarly, we can also derive $STK_{18}[1,5]$. Time complexity of this step could be approximated by $2^{25c} \cdot N \cdot 2^{-9c} \cdot 2 \cdot \frac{1}{21 \cdot 16} + 2^{27c} \cdot N \cdot 2^{-10c} \cdot 2 \cdot \frac{5}{21 \cdot 16} \approx N2^{17c-5.07}$.

5. *Satisfying the cell conditions in* ΔX_4. From the previous steps, we have known $STK_{20}[0,3,7]$, $STK_{18}[1,3,5]$ and $STK_0[5,6,7]$. With the tweakey schedule of SKINNY-n-3n, we can determine $STK_2[1,3,5]$ and thus compute $\Delta W_3[4,8,12]$. The conditions $\Delta W_3[4] = \Delta W_3[8] = \Delta W_3[12]$ will lead to two c-bit filters.
6. *Satisfying the cell conditions in* ΔW_{16}. Guess $STK_{18}[0,7]$ and compute $\Delta X_{17}[9,13]$. The condition $\Delta X_{17}[9] = \Delta X_{17}[13]$ would act as a c-bit filter. Also, we can derive $STK_{17}[1]$ with another condition $\Delta X_{17}[1] = \Delta X_{17}[13]$ by applying Lemma 2. Time complexity of this step is $2^{29c} \cdot N \cdot 2^{-12c} \cdot 2 \cdot \frac{2}{21 \cdot 16} = N2^{17c-6.39}$.
7. *Satisfying the cell conditions in* ΔX_4. Deduce $STK_2[1,7]$ from known $STK_{20}[0,1]$, $STK_{18}[1,7]$ and $STK_0[3,7]$. Guess $STK_2[2]$. Then we can compute $\Delta W_3[2,6,10]$. The conditions $\Delta W_3[2] = \Delta W_3[6] = \Delta W_3[10]$ will lead to two c-bit filters. Time complexity of this step is $2^{30c} \cdot N \cdot 2^{-13c} \cdot 2 \cdot \frac{1}{21 \cdot 16} = N2^{17c-8.39}$.
8. *Satisfying the cell conditions in* ΔX_5. Determine $STK_2[0]$ from $STK_0[1]$, $STK_{18}[0]$ and $STK_{20}[2]$. Guess $STK_2[6]$ and $STK_3[4]$. With the condition $\Delta X_5[11] = \Delta W_4[7] \oplus \Delta W_4[11] = \Delta Y_4[6] \oplus \Delta Y_4[9] = 0$, we can determine $X_4[6]$ and thus $STK_3[2]$ due to $X_4[6] = Y_3[2] \oplus STK_3[2]$. We can also determine $STK_3[0]$ from the knowledge of $STK_{19}[0]$, $STK_{17}[1]$ and $STK_1[1]$. The condition $\Delta W_4[11] = \Delta W_4[15]$ would act as a c-bit filter. Time complexity of this step is $2^{32c} \cdot N \cdot 2^{-15c} \cdot 2 \cdot \frac{3}{21 \cdot 16} = N2^{17c-6.81}$.

Complexity. The data complexity is $D = 2^{15c+\mathrm{LG}(g)+1}$. The time complexity of this whole attack is $2^{8c+15c+\mathrm{LG}(g)+1} \cdot \frac{8}{21 \cdot 16} + 2^{25c+17c+\mathrm{LG}(g)-3.81} + 2^{48c-g}$. For SKINNY-64-192 with $c = 4$, we set $g = 23$, then $D = 2^{64.99}$, $T = 2^{169.38}$, $M = 2^{103.99}$. For SKINNY-128-384 with $c = 8$, we set $g = 46$, then $D = 2^{125.99}$, $T = 2^{338.65}$, $M = 2^{204.99}$.

M Application to Midori64

M.1 Specification

Midori is a lightweight block cipher proposed by Banik *et al.* at ASIACRYPT 2015 [48]. Midori-family includes two ciphers: Midori64 with 64-bit block size (4-bit cell size) and 128-bit key size, Midori128 with 128-bit block size (8-bit cell size) and 128-bit key size.

Midori is a variant of substitution-permutation network (SPN) and the number of rounds is 16 for Midori64. The round function of Midori consists of four transformations: `SubCell` (`SB`), `ShuffleCell` (`SC`), `MixColumn` (`MC`) and `KeyAdd` (`AK`). In `SB` operation, a 4-bit S-box is applied to every 4-bit cell of the internal state S of Midori64. In `SC` operation, each 4-bit cell of the state S is permuted as follows:

$$(s_0, s_1, \ldots, s_{15}) \leftarrow (s_0, s_{10}, s_5, s_{15}, s_{14}, s_4, s_{11}, s_1, s_9, s_3, s_{12}, s_6, s_7, s_{13}, s_2, s_8).$$

The MC operation applies an involutory matrix

$$M = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

to every column of the state. In AK operation, the n-bit round key is XORed to the state S. The round function of Midori is shown in Figure 18.

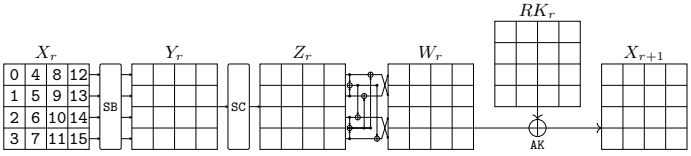


Fig. 18: Round function of Midori

Round Key Generation. For Midori64, a 128-bit secret key K is denoted as two 64-bit keys K_0 and K_1 as $K = K_0 \| K_1$. Then, $WK = K_0 \oplus K_1$ and $RK_i = K_{(i \bmod 2)} \oplus \alpha_i$, where $0 \leq i \leq 14$. The WK is used as the whitening key between the plaintext and round 0, and as the round key in the final round (where there is no SC and MC operations).

M.2 Single-Key Impossible Differential Attack on Midori64

This section provides a 11-round single-key impossible differential attack against Midori64. In this attack, we use the same 6-round single-key impossible differential as in [32]:

$$\Delta W_1 = (0000|0000|00?0|0000) \nrightarrow \Delta Z_7 = (0?00|0000|0000|0?00).$$

We prefix 2 rounds at the beginning and append 3 rounds at the end of the distinguisher to mount the attack, as shown in Figure 19. From the figure, we can get the parameters used for this attack: $r_{\mathrm{B}} = 9c$, $c_{\mathrm{B}} = 8c$, $r_{\mathrm{F}} = 14c$, $c_{\mathrm{F}} = 13c$, $|k_{\mathrm{B}} \cup k_{\mathrm{F}}| = 23c$, $c'_{\mathrm{B}} = 6c$, $c'_{\mathrm{F}} = 2c$, $|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| = 9c$. In the key-recovery extensions of this attack, it adopts deterministic extensions.

Pairs Collection. In this phase, we guess 2^{9c} possible values of $WK[0, 1, 4, 5, 6, 9, 10, 12, 14]$ to generate pairs. There will be $N = 2^{c^*_{\mathrm{B}} + c^*_{\mathrm{F}} + \mathrm{LG}(g)} = 2^{13c + \mathrm{LG}(g)}$ pairs need to be prepared.

Guess-and-Filter. For N pairs under each pre-guessed subkey bits:

Fig. 19: The single-key impossible differential attack against 11-round Midori64

1. *Satisfying the cell conditions in* ΔeW_9. With condition $\Delta eW_9[0] = \Delta X_{10}[1] \oplus \Delta X_{10}[2] = 0$, we can get the value of $\Delta X_{10}[2]$. By applying Lemma 2, we can deduce the value of $Y_{10}[2]$ and thus $WK[2]$. Similarly, we can derive $WK[7, 8, 13, 15]$. Time complexity of this step would be approximated by $2^{9c} \cdot N \cdot 2 \cdot \frac{5}{11 \cdot 16} = N2^{9c-4.14}$.
2. *Satisfying the cell conditions in* ΔeW_8. Guess $eRK_9[5, 11]$. The condition $\Delta X_9[4] = \Delta X_9[6]$ will lead to one c-bit filter. From the condition $\Delta X_9[4] = \Delta X_9[7]$, we can determine the value of $eRK_9[12]$. Guess $eRK_9[4, 13]$. The condition $\Delta X_9[13] = \Delta X_9[14]$ will lead to one c-bit filter. From the condition $\Delta X_9[13] = \Delta X_9[15]$, we can determine the value of $eRK_9[3]$. Time complexity of this step is $2^{11c} \cdot N \cdot 2 \cdot \frac{2}{11 \cdot 16} + 2^{13c} \cdot N \cdot 2^{-c} \cdot 2 \cdot \frac{2}{11 \cdot 16} = N2^{12c-5.46}$.
3. *Satisfying the cell conditions in* ΔZ_7. Due to $eRK_9[5] = K_1[4] \oplus K_1[6] \oplus K_1[7]$, $WK[4] = K_0[4] \oplus K_1[4]$,$WK[6] = K_0[6] \oplus K_1[6]$ and $WK[7] = K_0[7] \oplus K_1[7]$, we can compute $eRK_8[5] = K_0[4] \oplus K_0[6] \oplus K_0[7]$. Similarly, we can compute $eRK_8[12]$. The condition $\Delta X_8[4] = \Delta X_8[7]$ will lead to one c-bit filter. Time complexity of this step is $2^{13c} \cdot N \cdot 2^{-2c} \cdot 2 \cdot \frac{2}{11 \cdot 16} = N2^{11c-5.46}$.

4. *Satisfying the cell conditions in* ΔW_1. Guess $RK_0[3]$. We can derive $RK_0[6,9]$ by applying Lemma 2 with the condition $\Delta Y_1[3] = \Delta Y_1[6] = \Delta Y_1[9]$. Time complexity of this step is $2^{14c} \cdot N \cdot 2^{-3c} \cdot 2 \cdot \frac{3}{11 \cdot 16} = N2^{11c-4.87}$.

Complexity. The data complexity is $D = 2^{14c+\mathrm{LG}(g)+1}$. The time complexity is $2^{9c+14c+\mathrm{LG}(g)+1} \cdot \frac{9}{11 \cdot 16} + 2^{25c+\mathrm{LG}(g)-5.46} + 2^{32c-g}$. We set $g = 29$, then $D = 2^{61.33}$, $T = 2^{99.94}$, $M = 2^{56.33}$.

N Applications to Deoxys-BC

N.1 Specification

Deoxys-BC [49], the core primitive of authenticated encryption scheme Deoxys (winner of the CAESAR competition), is an 128-bit tweakable block cipher conforming to the `TWEAKEY` framework [50]. Deoxys-BC has two main versions: Deoxys-BC-256 with 256-bit tweakey size and Deoxys-BC-384 with 384-bit tweakey size.

Deoxys-BC takes an AES-like design and adopts a SPN structure that transforms the internal states through a round function similar to that of AES. Deoxys-BC-256 has 14 rounds, while Deoxys-BC-384 has 16 rounds.

The round function of Deoxys-BC consists of the four transformations in the order specified below:

- `AddRoundTweakey` (`ART`): XOR the 128-bit round subtweakey to the internal state.
- `SubBytes` (`SB`): Apply the 8-bit AES S-box $\mathcal{S}$ to the 16 bytes of the internal state.
- `ShiftRows` (`SR`): Rotate the 4-byte i-th row left by i positions, $i = 0, 1, 2, 3$.
- `MixColumns` (`MC`): Multiply the internal state by the 4×4 MDS matrix of AES.

At the end of the last round, a final `AddRoundTweakey` operation is applied to the internal state to produce the ciphertext. Figure 20 provides an overview of the round function of Deoxys-BC.

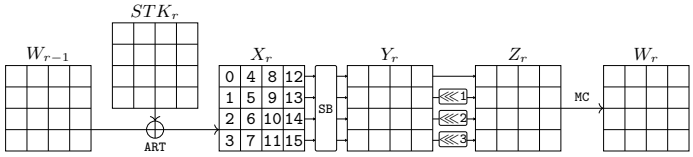


Fig. 20: Round function of Deoxys-BC

Tweakey Schedule. Different from the key schedule of AES, Deoxys-BC used a linear tweakey schedule under the `TWEAKEY` framework. We denote the concatenation of the key K and the tweak K as KT, i.e. $KT = K||T$. For Deoxys-BC-256, the size of

KT is 256 bits with the first (most significant) 128 bits denoted as W_1, the second W_2, while the 384 bits tweakey of Deoxys-BC-384 is divided into W_1, W_2 and W_3 per 128 bits sequentially. For Deoxys-BC-256, a subtweakey of i-th round is defined as $STK_i = TK_i^1 \oplus TK_i^2 \oplus RC_i$ while for the case of Deoxys-BC-384 it is defined as $STK_i = TK_i^1 \oplus TK_i^2 \oplus TK_i^3 \oplus RC_i$.

The 128-bit words TK_i^1, TK_i^2, TK_i^3 are outputs produced by tweakey schedule algorithm, initialized with $TK_0^1 = W_1$ and $TK_0^2 = W_2$ for Deoxys-BC-256 and with $TK_0^1 = W_1, TK_0^2 = W_2$ and $TK_0^3 = W_3$ for Deoxys-BC-384. The tweakey schedule algorithm is defined as

$$TK_{i+1}^1 = h(TK_i^1), TK_{i+1}^2 = h(LFSR_2(TK_i^2)), TK_{i+1}^3 = h(LFSR_3(TK_i^3)),$$

where the byte permutation h is defined as:

$$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 \\ 1 & 6 & 11 & 12 & 5 & 10 & 15 & 0 & 9 & 14 & 3 & 4 & 13 & 2 & 7 & 8 \end{pmatrix}.$$

The $LFSR_2$ and $LFSR_3$ functions are the application of an LFSR to each of the 16 bytes of a tweakey 128-bit word. The two LFSRs used are given in Table 9.

Table 9: Two LFSRs used in Deoxys-BC tweakey schedule

$LFSR_2$	$(x_7 \|\|x_6\|\| x_5 \|\|x_4\|\| x_3 \|\|x_2\|\| x_1\|\|x_0) \rightarrow (x_6 \|\|x_5\|\| x_4 \|\|x_3\|\| x_2 \|\|x_1\|\| x_0\|\|x_7 \oplus x_5)$
$LFSR_3$	$(x_7 \|\|x_6\|\| x_5 \|\|x_4\|\| x_3 \|\|x_2\|\| x_1\|\|x_0) \rightarrow (x_0 \oplus x_6 \|\|x_7\|\| x_6 \|\|x_5\|\| x_4 \|\|x_3\|\| x_2\|\|x_1)$

For more details on the specification of Deoxys-BC, please refer to [49].

N.2 Related-Tweakey Impossible Boomerang Attack on Deoxys-BC-256

In this section, we propose a new 10-round related-tweakey impossible boomerang attack against Deoxys-BC-256. In this attack, we use a 7-round related-tweakey impossible boomerang distinguisher, prefix one round at the beginning and append 2 rounds at the end of the distinguisher to mount the attack, as shown in Figure 21. From the figure, we can get the parameters used for this attack: $r_{\mathrm{B}} = 2c$, $c_{\mathrm{B}} = 2c$, $r_{\mathrm{F}} = 9c$, $c_{\mathrm{F}} = 9c$, $|k_{\mathrm{B}} \cup k_{\mathrm{F}}| = 13c$, $c'_{\mathrm{B}} = 2c$, $c'_{\mathrm{F}} = 3c$, $|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| = 6c$. In the key-recovery extensions of this attack, it adopts deterministic extensions.

Quartets Collection. In this phase, we guess 2^{6c} possible values of $STK_{10}[3, 6, 9, 12]$ and $STK_0[8, 13]$ to generate quartets. There will be $Q = 2^{2c_{\mathrm{B}}^* + 2c_{\mathrm{F}}^* + \mathrm{LG}(g)} = 2^{12c + \mathrm{LG}(g)}$ quartets need to be prepared.

Guess-and-Filter. For Q quartets under each pre-guessed subkey bits:

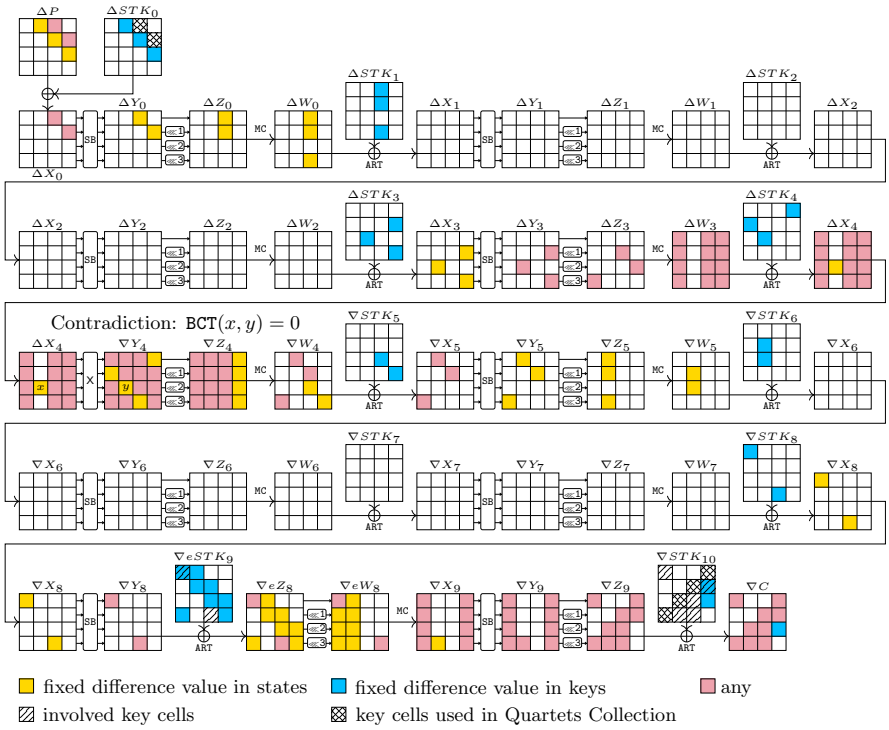


Fig. 21: The related-tweakey impossible boomerang attack against 10-round Deoxys-BC-256

1. *Satisfying the cell conditions in* ∇X_9. Guess $STK_{10}[11]$. The condition with known $\nabla X_9[7]$ on both sides of the boomerang will lead to two c-bit filters. Time complexity of this step is $2^{7c} \cdot Q \cdot 4 \cdot \frac{1}{10 \cdot 16} = Q2^{7c-5.32}$.
2. *Satisfying the cell conditions in* ∇X_8. Guess $eSTK_9[11]$ and compute $\nabla X_8[11]$. The condition with known $\nabla X_8[11]$ will lead to two c-bit filters. Time complexity of this step is $2^{8c} \cdot Q \cdot 2^{-2c} \cdot 4 \cdot \frac{1}{10 \cdot 16} = Q2^{6c-5.32}$.
3. *Satisfying the cell conditions in* ∇X_8. Guess $STK_{10}[0, 7, 10, 13]$. The condition $\Delta eW_8[12-14] = 0$ will lead to six c-bit filter. Guess $eSTK_9[0]$. The condition with known $\nabla X_8[0]$ on both sides of the boomerang will lead to two c-bit filters. Time complexity of this step is $2^{12c} \cdot Q \cdot 2^{-4c} \cdot 4 \cdot \frac{4}{10 \cdot 16} = Q2^{8c-3.32}$.

Complexity. The data complexity is $D = 2^{16c+\frac{\mathrm{LG}(g)}{2}+2}$. The time complexity is $2^{6c+16c+\frac{\mathrm{LG}(g)}{2}+2} \cdot \frac{6}{10 \cdot 16} + 2^{6c+16c+\frac{\mathrm{LG}(g)}{2}+1} \cdot \frac{6}{10 \cdot 16} + 2^{20c+\mathrm{LG}(g)-3.32} + 2^{32c-g}$. We set $g = 80$, then $D = 2^{132.9}$, $T = 2^{177.42}$, $M = 2^{56}$.

N.3 Related-Tweakey Impossible Boomerang Attack on Deoxys-BC-384

In this section, we propose a new 14-round related-tweakey impossible boomerang attack against Deoxys-BC-384. In this attack, we use a 9-round related-tweakey impossible boomerang distinguisher, prefix 3 rounds at the beginning and append 2 rounds at the end of the distinguisher to mount the attack, as shown in Figure 22. From the figure, we can get the parameters used for this attack: $r_{\mathrm{B}} = 16c$, $c_{\mathrm{B}} = 16c$, $r_{\mathrm{F}} = 6c$, $c_{\mathrm{F}} = 6c$, $|k_{\mathrm{B}} \cup k_{\mathrm{F}}| = 35c$, $c'_{\mathrm{B}} = 8c$, $c'_{\mathrm{F}} = 6c$, $|k'_{\mathrm{B}} \cup k'_{\mathrm{F}}| = 26c$. In the key-recovery extensions of this attack, it adopts deterministic extensions.

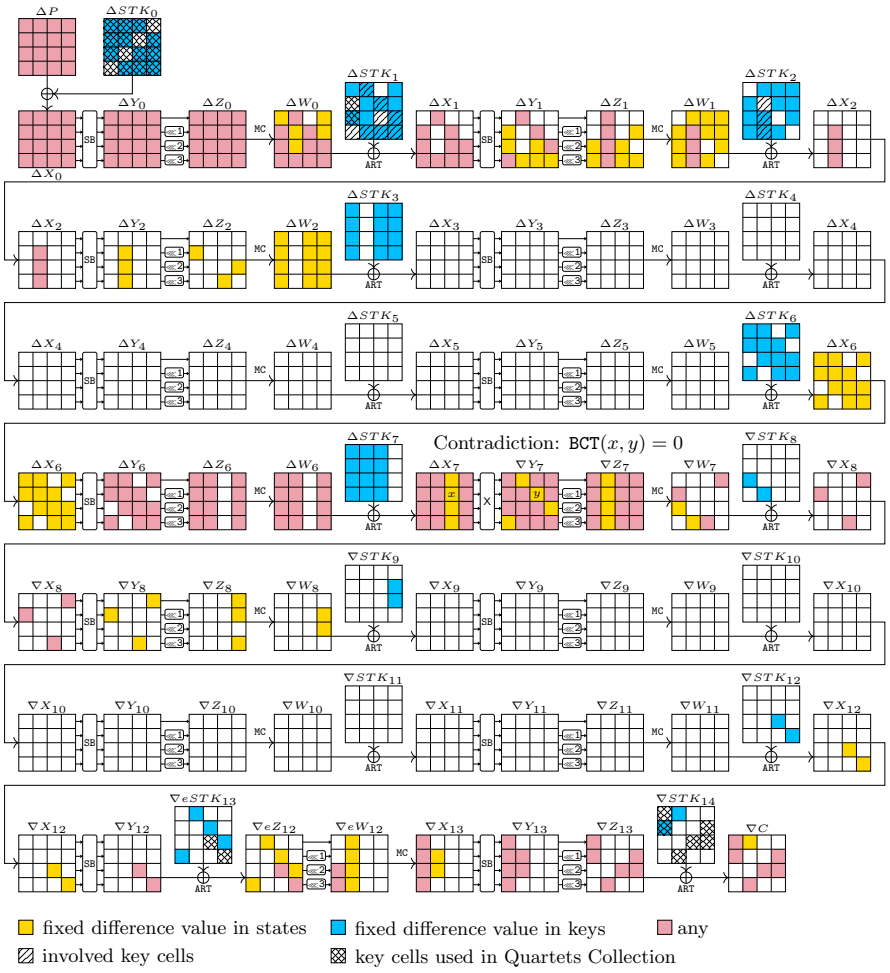


Fig. 22: The related-tweakey impossible boomerang attack against 14-round Deoxys-BC-384

Quartets Collection. In this phase, we guess 2^{26c} possible values of $STK_0[0-15]$, $STK_1[1-2]$, $STK_{14}[0,1,7,10,13,14]$ and $eSTK_{13}[11,15]$ to generate quartets. There will be $Q = 2^{2c_{\text{B}}^* + 2c_{\text{F}}^* + \text{LG}(g)} = 2^{16c+\text{LG}(g)}$ quartets need to be prepared.

Guess-and-Filter. For Q quartets under each pre-guessed subkey bits:

1. *Satisfying the cell conditions in* ΔY_1. Guess $STK_1[7]$ and compute $\Delta Y_1[7]$. The condition with known $\Delta Y_1[7]$ will lead to two c-bit filters. Similarly, the conditions with known $\Delta Y_1[10,11,15]$ will lead to six c-bit filters by guessing $STK_1[10,11,15]$. Time complexity of this step would be approximated by $2^{27c} \cdot Q \cdot 4 \cdot \frac{1}{14 \cdot 16} = Q2^{27c-5.81}$.
2. *Satisfying the cell conditions in* ΔW_1. Guess $STK_1[3,4,9,14]$. The condition with known $\Delta W_1[4]$ on both sides of the boomerang will lead to two c-bit filters. Time complexity of this step is $2^{34c} \cdot Q \cdot 2^{-8c} \cdot 4 \cdot \frac{4}{14 \cdot 16} = Q2^{26c-3.81}$.
3. *Satisfying the cell conditions in* ΔY_2. We can determine $STK_2[5,6]$ from known $STK_0[3,8]$, $STK_1[10,15]$ and $STK_{14}[13,14]$ by applying the tweakey schedule of Deoxys-BC-384. Guess $STK_2[7]$. The conditions $\Delta Y_2[5-7] = 0$ will lead to six c-bit filter. Time complexity of this step would be a negligible one compared to previous steps.

Complexity. The data complexity is $D = 2^{16c+\frac{LG(g)}{2}+2}$. The time complexity of the whole attack is $2^{26c+16c+\frac{LG(g)}{2}+2} \cdot \frac{26}{14 \cdot 16} + 2^{26c+16c+LG(g)+1} \cdot \frac{26}{14 \cdot 16} + 2^{27c+16c+LG(g)-5.81} + 2^{48c-g}$. We set $g = 41$, then $D = 2^{132.41}$, $T = 2^{343.05}$, $M = 2^{72}$.